The University of Hong Kong

School of Computing and Data Science

COMP7705

Project Report

Semantic-Motion: A Multi-view Engineering Pipeline for Perception, Fact-preserving Language Augmentation and Fail-closed Multimodal Refinement

Submitted in partial fulfillment of the requirements for the admission to the degree of Master of Science in Computer Science

By

Gai Yifan(), Yang Tongzhou(3036657718), Li Chenyang (3036657835)

Mentor: Prof. Taku Komura

Date of submission: 17/07/2026

Declaration

We declare that this report and the engineering work it describes are our own, produced for the COMP7705 project, and that all sources, datasets, software libraries and open-source components used have been acknowledged and cited. Where an external open-source component was integrated into the pipeline, it is treated as a replaceable module behind our own interfaces and is credited in the references. No part of this report has been submitted for any other qualification.

In accordance with academic-integrity requirements, we disclose that generative AI tools were used to assist with drafting and editing prose, and with routine coding and refactoring tasks. All AI-assisted output was reviewed, corrected and validated by the authors, who take full responsibility for the final content, the implementation and the reported results.

This is a group project undertaken by Gai Yifan, Yang Tongzhou and Li Chenyang. All authors jointly contributed to the overall system design, the integration of the four modules, the evaluation methodology and the writing of this report, with each author taking primary responsibility for a share of the module implementation and experiments as agreed within the group.

Abstract

Recent vision-language-action (VLA) systems require large corpora of aligned demonstrations, yet raw loco-manipulation motion capture and video do not map cleanly onto the inputs a vision-language model consumes. This report presents Semantic-Motion, an end-to-end engineering pipeline that converts motion or simulation data into auditable Video–Motion–Text samples through four file-linked modules: a Blender rendering module that produces synchronized fixed and egocentric videos with 3D trajectories; a paired-view perception module that segments video into macro intents and micro instructions using overlapping windows and weighted boundary voting; a language augmentation module that rewrites instructions with source-step provenance; and a fail-closed refinement module that scores physical motion quality and independently verifies text–video consistency. The evaluated outputs are structured task records, motion-quality diagnostics and explainable keep/drop decisions. On controlled motion corruptions the quality detector ranks clean against corrupted trajectories with an AUROC of 1.0 on the calibration track, and on a weak LIBERO proxy fused input improves title token F1 over a fixed view by +0.055 (95% CI [+0.020, +0.089]). The main limitation is that formal boundary and keep/drop metrics await human-adjudicated gold; augmentation fact-checking and refinement gates are deliberately disabled, and semantic perception depends on an external API.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Project Context: From Motion Data to VLA-ready Multimodal Data

Recent vision-language-action (VLA) systems seek to map visual observations and natural-language instructions directly to robot actions. Representative systems such as RT-2 encode actions as tokens and co-train on web-scale vision-language data and robot trajectories [11], while OpenVLA provides an open 7-billion-parameter implementation trained on a large and diverse collection of robot demonstrations [3]. These developments make the availability of aligned multimodal training examples increasingly important. A useful demonstration is no longer only a video or a joint trajectory; it is a traceable tuple linking what was observed, how the body or robot moved, what task was performed, and which temporal interval supports the description.

The motivating data source for this project is loco-manipulation motion: whole-body movement in which locomotion, posture change, reaching and interaction may occur in the same sequence. The Master Proposal identifies a representation gap between such motion data and the inputs expected by vision-language systems. Raw FBX/BVH animation and frame-wise 3D joint coordinates are useful for kinematic analysis, but a general-purpose VLM does not directly consume these representations. Conversely, video alone hides the precise trajectory information required to assess smoothness, acceleration, jerk and temporal coverage. The project therefore treats video, motion and language as complementary views of one sample rather than as interchangeable data products.

This motivation is consistent with the broader move towards heterogeneous robot data. Open X-Embodiment aggregates data from many institutions and embodiments to study cross-robot transfer [8], while Being-H0.5 explicitly investigates human-centric pre-training and cross-embodiment generalisation [5]. Semantic-Motion does not attempt to reproduce those large policy-training systems. It addresses an earlier engineering question: how can raw or rendered demonstrations be converted into auditable *candidate* Video–Motion–Text records before they are admitted to a VLA training corpus?

The implemented system answers this question through four modules. In operational order, Module D renders motion into synchronized fixed and egocentric videos and exports 3D trajectories; Module A extracts temporally grounded macro-intents and micro-instructions; Module B rewrites those instructions into language variants; and Module C reconstructs standardized samples and evaluates physical motion quality and text–video consistency. The modules are joined by explicit JSON/JSONL interfaces and are exposed through both command-line entry points and a Web-based orchestration dashboard.

1.2 Engineering Problem and Motivation

The central engineering problem is not simply video captioning. It is the construction of a repeatable data transformation in which evidence remains aligned across camera views, time, motion and generated language. Five coupled failure sources make this difficult.

1. **Representation mismatch.** Motion-capture files and simulation trajectories are not directly

interpretable by a VLM, whereas a rendered video does not preserve exact physical derivatives.

2. **Temporal ambiguity.** A long demonstration can contain several tasks and transitions. Uniformly assigning one caption to the whole video can merge unrelated actions, while isolated frame captions lose action order and task boundaries.
3. **View-dependent evidence.** A fixed camera is effective for whole-body displacement and scene geometry, but an egocentric view can better expose local interaction. Treating either view as complete can produce unsupported semantic claims.
4. **Generative uncertainty.** VLM annotations and LLM rewrites can omit actions, invent objects, reverse temporal order, or add plausible but unobserved details.
5. **Silent data-quality failure.** A linguistically plausible description may be paired with a noisy or incomplete trajectory. Conversely, a smooth trajectory does not guarantee that the generated text describes the corresponding video.

Large demonstration datasets illustrate why scalable processing matters. BridgeData V2 collects diverse manipulation trajectories across multiple environments [10]; MimicGen shows how a small number of human demonstrations can be transformed into much larger simulated datasets [6]. At such scales, manual inspection of every Video–Motion–Text tuple is impractical. The required solution is therefore a modular and observable pipeline that rejects malformed interfaces deterministically, attaches machine-readable reasons to suspicious samples, and retains enough provenance for later human audit.

1.3 Project Aim and Engineering Deliverables

The project aim is to design and implement an end-to-end engineering pipeline that converts paired visual and physical motion evidence into structured, language-annotated demonstration samples and evaluates whether those samples are suitable candidates for downstream VLA research.

The intended deliverables are:

- a Blender-based rendering workflow for FBX/BVH motion that produces synchronized `Fixed_View.mp4` and `Ego_View.mp4` streams together with a real 3D trajectory;
- a manifest generator that assigns a stable `sample_id` and joins each pair of views to its trajectory;
- a paired-view semantic perception pipeline with overlapping macro windows, weighted transition aggregation, dense micro windows and structured task records;
- a language augmentation interface supporting source-only, deterministic template and API-backed LLM rewriting modes, with provenance and audit metadata;
- a Module C preparation and refinement workflow that normalizes motion tracks, verifies cross-modal alignment, computes motion-quality indicators, performs multi-view semantic verification and emits decisions with stable reason codes;
- independent per-sample JSON and JSONL artifacts, avoiding hidden concatenation or filename collisions;
- a FastAPI and React/Vite control surface for asynchronous execution, progress polling, captured subprocess logs and visualization of Motion Quality, Semantic Consistency, Keep/Discard decisions and reason codes; and
- tests and explicit metadata sufficient to distinguish automatically verified behavior from API-dependent, Blender-dependent and human-annotation-dependent evidence.

1.4 System Requirements

The requirements below are phrased as engineering acceptance criteria. Where the present implementation only partially meets a criterion, the limitation is stated explicitly rather than treating planned work as completed.

Functional requirements

FR1: Rendered evidence For each valid source motion, the rendering stage shall produce fixed and egocentric videos plus a trajectory containing frame-wise positions for configured tracks such as Root, Hand_R and Hand_L.

FR2: Manifest indexing Each sample shall have a unique identifier, two resolvable view paths and a resolvable trajectory path. Invalid directories shall be reported rather than silently indexed.

FR3: Shared temporal contract All views shall be represented against one FPS, frame count, duration and frame map. Perception shall sample identical source-frame indices from selected views.

FR4: Multi-granularity perception The system shall represent high-level task intent, temporally ordered micro-instructions, objects, spatial relations, body part, contact state, posture, confidence and evidence-frame references.

FR5: Controlled augmentation Every generated instruction variant shall retain its source-step references, rewriter identity, model and prompt provenance. The design objective is fact-preserving augmentation. The current API-backed rewriter relies on prompt constraints and records `validation=disabled`; a deterministic post-generation fact gate remains an acceptance gap.

FR6: Multimodal refinement Module C shall load the real trajectory, normalize supported spatial units, check paired-view/time/motion coverage, score motion tracks, and independently judge text–video consistency.

FR7: Explainable output Refinement shall emit `motion_quality_score`, `semantic_label`, `semantic_confidence`, `decision`, `reason_codes` and diagnostic auxiliary data.

FR8: Independent artifacts Perception, prepared-sample and refined outputs shall be written under sample-specific filenames.

FR9: Selective and batch execution The orchestrator shall process all manifest entries by default and optionally restrict execution to a validated `target_sample_id`.

FR10: Observability An operator shall be able to start a task, poll its state, inspect stage progress and logs, and retrieve available refined results without blocking the browser request for the duration of model inference.

Non-functional requirements

NFR1: Traceability Outputs shall preserve sample identifiers, source paths, dataset identity, generator, Git revision, evidence intervals and configuration metadata where available.

NFR2: Modularity Rendering, perception, augmentation and refinement shall remain callable as separate stages with file-based contracts.

NFR3: Robust failure reporting Missing files, unsupported units, unreadable videos, subprocess failures and API failures shall be surfaced through exceptions, task status or reason codes.

NFR4: Reproducibility Deterministic windowing and scoring shall be configuration-driven.

External VLM/LLM responses must be marked as model- and API-dependent.

NFR5: Resource accessibility The semantic stages shall be runnable without local model training by using an OpenAI-compatible cloud endpoint, while acknowledging the resulting cost and reproducibility trade-offs.

1.5 Scope and Boundaries

This section defines what Semantic-Motion delivers in the present project and what remains outside the scope of its engineering remit. It is necessary to clarify these boundaries because the process is closely linked to several related research areas, such as foundational vision-language models, robotic policy learning and physical deployment, none of which form part of the main deliverables of this work.

The project covers the design, implementation, and evaluation of an end-to-end pipeline of data engineering. This pipeline converts motion or simulation data into aligned video, motion, text samples with quality control. Four modules constitute the core of this pipeline, each of which is is scoped to concrete engineering contracts rather than to open model research.

For each valid source motion, the rendering stage produces Fixed View and Ego View videos together with a trajectory that records positions for configured tracks such as Root, Hand_R and Hand_L. Each sample is indexed by a unique identifier with resolvable view paths and a resolvable trajectory path.

All views of a sample share one FPS, frame count, duration, and frame map. Perception module samples identical indices of source frames from the selected views. From these observations the system represents high-level task intent, temporally ordered micro-instructions, objects, spatial relations, body part, contact state and posture. Every generated instruction variant retains its reference of source steps, together with rewriter identity, model, and prompt provenance. The design objective is fact-preserving augmentation.

The refinement module loads the real trajectory, normalizes supported spatial units, checks paired-view, time, and motion coverage, scores motion tracks, and judges text-video consistency. Refinement emits diagnostic auxiliary data which helps operators to audit why a sample was kept or dropped.

Traceability requires that outputs preserve sample identifiers, source paths, dataset identity, generator and evidence intervals. Modularity requires that rendering, perception, augmentation, and refinement remain callable as separate stages with file-based contracts. Robust failure reporting surfaces missing files, unsupported units, unreadable videos, subprocess failures, and API failures through exceptions, task status, or reason codes. Deterministic windowing and scoring are driven by configuration. External VLM/LLM responses are dependent on model and API. The semantic stage can be run via an OpenAI-compatible cloud endpoint without the need for local model training, and the trade-off between associated costs and reproducibility has been incorporated into the engineering scope.

Several adjacent capabilities are deliberately excluded so that the project remains focused on data infrastructure rather than on model training or robot control.

First, the project does not train a new foundation vision-language model, nor does it design or train a complete VLA policy. Existing VLM APIs are consumed as interchangeable perception backends. Improving their parameters or architectures is outside the present contribution.

Second, the system does not predict robot control signals such as joint torques, motor commands, or closed-loop actions. Its outputs are multimodal training samples and quality decisions, not executable policies.

Third, the project does not include closed-loop deployment of real robots. Though physical embodiment gaps have been acknowledged in relevant contexts, such as the discrepancy between the skeletal movements of the humanoid robot by rendering and the trajectory of the manipulator's end-effector, bridging these gaps through hardware experiments does not fall within the scope of the deliverables.

In summary, Semantic-Motion is scoped as a modular, observable, and extensible engineering pipeline for producing aligned video-motion-text data. Claims of contribution are limited to what has been implemented, integrated, and demonstrated within these boundaries.

1.6 Contributions

The principal contribution is a runnable and inspectable interface between motion rendering, video semantics, language generation and quality control. More specifically, the project contributes:

1. a versioned `ViewBundle` contract that links synchronized camera streams, a shared clock, a motion reference and provenance;
2. a paired-view temporal perception implementation that interleaves fixed/ego evidence at common timestamps, combines overlapping macro-window transition votes using Hanning weights, and derives dense micro-level descriptions;
3. a common augmentation interface with explicit source-only, template and LLM-backed modes and auditable rewrite metadata;
4. a standardized Module C schema and multi-track motion-quality calculation that exposes velocity, acceleration, jerk, jitter and temporal regularity diagnostics;
5. joint multi-view semantic verification and machine-readable explanations for refinement outcomes;
6. a per-sample automation layer that validates manifest paths, preserves independent artifacts and records subprocess output; and
7. a Web dashboard that exposes orchestration state and final evaluation dimensions without replacing the underlying command-line interfaces.

These contributions are primarily architectural and engineering contributions. The report does not claim a new foundation model, a new policy-learning algorithm, or statistically established superiority over large-scale VLA systems.

1.7 Report Organisation

The remainder of this report is organised according to the engineering life cycle of the Semantic-Motion pipeline, from technical background and requirements through design, integration, evaluation and reflection.

Chapter 2 reviews the technical background and related systems that motivate the project. It introduces Vision-Language-Action systems and the role of vision-language models as their perception foundation, surveys robot-learning benchmarks such as LIBERO and RoboCasa, discusses embodied vision-language models for object recognition, spatial reasoning and task inference, and examines video-to-task generation together with demonstration-quality issues such as semantic inconsistency,

hallucinated descriptions, incorrect temporal boundaries and motion anomalies.

Chapter 3 derives the system architecture and interface contracts from the requirements established in Chapter 1. It specifies inputs, outputs and use cases, states the design goals of modularity, configurability, interpretability and resource accessibility, and presents the overall four-module architecture along the data flow from motion or simulation assets to Video-Motion-Text samples with quality control. Module responsibilities and boundaries, shared data models and file-based interface contracts, observability and visualisation design, and the end-to-end execution workflow are defined so that later implementation and evaluation chapters share a common engineering vocabulary.

Chapter 4 describes the design and implementation of the four modules in data-flow order. The Rendering Module converts FBX/BVH or related motion assets into Fixed View and Ego View videos and exports three-dimensional trajectories. The Perception Module extracts structured task semantics from visual observations. The Augmentation Module produces textual variants from those semantics. The Refinement Module jointly assesses motion quality and text–video consistency before emitting an explainable keep/drop decision. Each module is presented through its inputs, processing logic, outputs, failure modes and replaceable interfaces.

Chapter 5 reports integration and evaluation evidence. It verifies module interfaces and shared temporal contracts, presents end-to-end demonstrations with observable intermediate products, and evaluates the system through video case studies, cross-module analyses and component or configuration studies.

Chapter 6 discusses the engineering strengths of the pipeline, analyses failure modes observed in practice, and reflects on limitations and trade-offs, including embodiment gaps, API-dependent perception and evaluation validity. It also considers implications for future VLA-oriented data pipelines. Chapter 7 concludes by summarizing the delivered engineering outcomes and outlining future work, such as stronger factual validation of augmented text, broader dataset coverage and closer coupling to downstream learning workflows.

Chapter 2

Technical Background and Related Systems

2.1 Vision-Language-Action Models

VLA models extend vision-language reasoning to action generation by conditioning a policy on observations and language. RT-2 demonstrates one influential approach: robot actions are discretized into text tokens so that a vision-language model can be co-fine-tuned on robot trajectories and web data [11]. OpenVLA instead emphasizes accessibility, releasing a 7B-parameter VLA and training infrastructure based on a large collection of real robot demonstrations [3]. Open X-Embodiment further shows the scale and heterogeneity of data required to study transfer across institutions and robot embodiments [8].

These systems motivate Semantic-Motion in two ways. First, language-conditioned control assumes that instructions are grounded in the corresponding visual and physical sequence. Second, pooling data across sources increases the importance of explicit identifiers, coordinate conventions, timebases and provenance. Semantic-Motion therefore operates upstream of policy learning: it creates and checks candidate multimodal records but does not generate control actions.

Being-H0.5 is particularly relevant to the proposal because it frames human-centric data as a resource for cross-embodiment transfer [5]. The present project adopts the motivation, but not the claimed scale or policy architecture. Human skeletal trajectories rendered in Blender remain separated from robot action spaces, and the embodiment gap is a core limitation rather than an assumed solved problem.

2.2 Robot Learning Benchmarks

LIBERO provides 130 procedurally generated manipulation tasks organized into four suites and is designed to study declarative and procedural knowledge transfer in lifelong robot learning [4]. RoboCasa supplies a large-scale household simulation platform with diverse tasks, scenes and objects [7]. Both are useful references for task categories and structured demonstrations, but neither directly supplies gold labels for the temporal micro-actions and semantic consistency decisions defined by this project.

Semantic-Motion uses benchmark resources at two levels. A small synthetic image benchmark supports static tests of object recognition, task classification, action sequence, semantic similarity and spatial reasoning. Separately, selected LIBERO Goal demonstrations can be converted into paired agent-view and eye-in-hand videos with end-effector trajectories. The latter are more representative of the system's contracts, but the BDDL task title is only a weak video-level label. It cannot validate contact state, precise temporal boundaries or refinement decisions without additional human annotation.

Open X-Embodiment [8] and BridgeData V2 [10] provide a broader data-engineering context. Their diversity demonstrates the value of normalized interfaces, but the current project does not claim

direct ingestion of all their native formats. Its implemented adapters cover Module D trajectories, selected LIBERO exports and standard single- or multi-track motion JSON.

2.3 Vision-Language Models for Robotics

VLMs provide a practical interface between visual evidence and structured language. PaLM-E interleaves continuous sensor representations and text within an embodied language model and demonstrates transfer across embodied reasoning, visual question answering and planning tasks [2]. RT-2 uses vision-language pre-training more directly for action prediction [11]. Semantic-Motion uses the same general grounding capability for a narrower purpose: observation and verification.

In Module A, a Qwen-compatible VLM receives sampled frames and is prompted to return objects, spatial relations, task type, action order, transition indices, body part, posture and contact information. In Module C, an independently configured verifier receives the candidate text together with one or both views and returns a normalized label, confidence and error types. This separation is important: generation confidence is not treated as proof that a description is faithful.

However, an API response is neither deterministic nor self-validating. Model updates, sampling, image compression, prompt changes and service failures can alter outputs. The system consequently stores raw responses and model/prompt metadata where possible and distinguishes deterministic schema checks from model-mediated judgments.

2.4 Video-to-Task Generation

Long demonstration videos create a joint segmentation and labelling problem. Video2Tasks is an open-source system that processes overlapping windows, uses a VLM to propose task transitions and instructions, and aggregates transition evidence with Hanning weights [9]. It is a software baseline rather than a peer-reviewed benchmark, so comparisons must pin the upstream revision and invoke its actual algorithmic functions rather than compare prompt snippets alone.

Semantic-Motion adopts the overlapping-window principle but extends the data contract. Macro windows propose task boundaries and high-level intent; dense micro windows describe shorter action primitives within each accepted segment. For paired input, frames from fixed and ego views are extracted at identical source indices and interleaved by timestamp before inference. Segment records retain frame intervals, evidence by view, macro intent, micro-instructions, trajectory references and augmentation metadata. This richer output is needed for later synchronization and semantic checks.

2.5 Demonstration Data Quality

Demonstration scale does not remove the need for quality control. BridgeData V2 shows the value of task and environment diversity for scalable robot learning [10], and MimicGen demonstrates how object-centric subtask structure can expand a small set of demonstrations into a larger dataset [6]. Diffusion Policy further illustrates that modern visuomotor policies learn distributions over temporally structured actions [1]; corrupted timing or physically implausible trajectories are therefore not benign metadata defects.

For this project, quality has at least three dimensions. *Contract quality* concerns file availability, paired-view synchronization, coordinate units and motion coverage. *Physical quality* concerns velocity,

acceleration, jerk, directional oscillation and timestamp regularity. *Semantic quality* concerns whether the text identifies the observed actors, objects, task and temporal order without hallucination. A useful filter must keep these dimensions separate in diagnostics even if it ultimately emits a binary decision.

2.6 Engineering Gap

Existing systems tend to optimize one part of the chain. VLA models focus on policy learning; benchmark suites focus on standardized tasks; scalable data generation systems focus on obtaining more trajectories; and video-to-task tools focus on segmentation and instruction generation. The project materials identify a gap between these components: there is no lightweight, local, auditable path that starts with human or simulated motion, creates synchronized multi-view evidence, generates multi-granularity language, attaches real motion and returns explainable quality decisions.

Semantic-Motion addresses this gap through explicit file and schema contracts rather than a monolithic model. The resulting design can be tested stage by stage, can retain independent artifacts for audit, and can expose failure reasons to both scripts and a Web dashboard. Its remaining gap is equally important: prompt-constrained augmentation is not yet deterministically fact-checked, and not every recorded synchronization or confidence diagnostic currently changes the final decision. Chapters 3 and 4 therefore distinguish the target fail-closed architecture from the precise behavior of the present implementation.

Chapter 3

System Requirements and Overall Design

3.1 Inputs, Outputs and Use Cases

The system accepts three principal source forms:

1. Module D directories containing the required fixed-view video, ego-view video and sample-specific trajectory;
2. a versioned `ViewBundle` or dataset manifest referencing paired views and a trajectory; and
3. compatibility inputs consisting of a single video or a directory of pre-extracted frames, which are restricted to debugging unless paired-view requirements are explicitly relaxed.

For each manifest sample, the automated path writes three independent artifact pairs: a perception JSON, a machine-readable sample JSONL with a pretty JSON equivalent, and a refined JSONL with a pretty JSON equivalent. The refined record exposes the dimensions required by the dashboard: `motion_quality_score`, `semantic_label`, `semantic_confidence`, `decision`, `reason_codes` and nested diagnostics.

The main use cases are:

- **Batch dataset preparation**: process every sample in a Module D manifest while retaining independent outputs;
- **Targeted debugging**: execute one `target_sample_id` and inspect its intermediate artifacts;
- **Controlled view study**: run fixed, ego and fused perception modes for ablation;
- **Quality review**: sort or inspect samples by motion score, semantic label, decision and reason code;
- **Regression evaluation**: run deterministic contract tests, motion corruptions and, where gold labels exist, temporal and decision metrics; and
- **Operator monitoring**: launch the pipeline through the Web interface and poll progress without holding one synchronous HTTP request open.

3.2 Design Goals

The architecture is guided by six design goals.

Evidence alignment. Fixed video, ego video, motion and text must refer to one sample and one temporal interval.

Explicit contracts. Cross-module communication must use versioned, validated models or documented JSON/JSONL fields rather than implicit filename assumptions alone.

Independent auditability. Each processing stage must retain its own per-sample artifact so that a final decision can be traced backwards.

Conservative quality control. Missing physical or semantic evidence should be visible and should ultimately lead to fail-closed policy behavior. The current decision wiring is assessed against this target.

Replaceable components. Recognition models, instruction rewriters, semantic verifiers and motion

sources must be replaceable behind narrow interfaces.

Operational observability. Long-running external API calls must expose status, progress, current stage and captured logs.

3.3 Overall Architecture

The architecture is a staged transformation with a feedback interpretation rather than an online control loop. Module C reason codes can inform later changes to prompts, rendering and thresholds, but they do not automatically modify upstream models during one execution.

Figure 3.1 shows the end-to-end data flow and the principal artifacts exchanged between the four modules; Table 3.1 then details each stage.

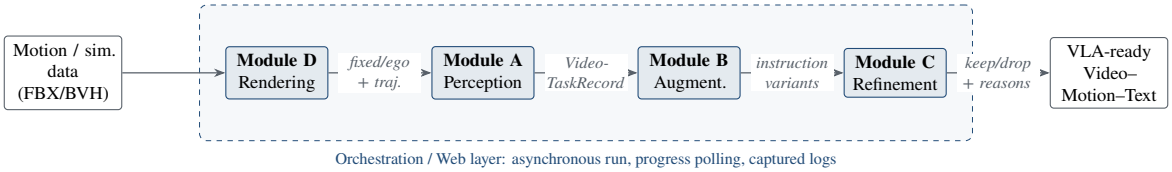

Figure 3.1. End-to-end four-module Semantic-Motion pipeline and the artifacts exchanged between stages. Module D renders motion into synchronized views and a trajectory; Module A perceives temporally grounded task records; Module B adds instruction variants; and Module C emits explainable keep/drop decisions. The dashed region marks the shared orchestration layer.

The Python process remains the system of record. The React dashboard does not recompute scientific metrics; it normalizes scores for display and maps `keep/drop` to operator-facing labels.

3.4 Module Responsibilities and Boundaries

The practical Module A/B boundary is an in-process boundary: `VideoTaskPipeline` owns a `PerceptionStream` and an `AugmentationStream`, and `run_video_task.py` serializes their combined result. The Module C boundary is deliberately file-based so that perception records can be inspected or replaced before refinement.

3.5 Data Models and Interface Contracts

Pydantic models in `src/semantic_motion/models.py` define the upstream contracts, while dataclasses in `src/module_c/schema.py` define the refinement input and output. Every contract carries a stable `sample_id`; temporal segments add start/end frames and seconds; and provenance records the dataset, source, generator and Git revision. JSONL is used for streaming and machine processing, while pretty JSON is written as an equivalent inspection artifact.

3.5.1 Paired Multi-view Video Contract

A formal sample contains at least the keys `fixed` and `ego`. Each maps to a `ViewStream` with a path, camera type, FPS, frame count and duration. `build_view_bundle` probes videos rather than trusting filenames, constructs a shared timebase from a reference stream, and records validation diagnostics for mismatched metadata. During fused perception, identical source-frame indices are extracted from

Table 3.1. End-to-end architecture and principal artifacts.

Stage	Primary implementation	Transformation	Principal output
Rendering (D)	`module_d/render_pipeline.py`	Imports FBX/BVH motion, constructs scene context, renders fixed/ego video and exports selected bone trajectories.	Two MP4 files and one trajectory JSON per action.
Indexing	`tools/prepare_module_d_manifest.py`	Validates expected files and creates portable sample references.	`data/module_d_output/manifest.json`.
Perception (A)	`src/semantic_motion/video_task.py`	Performs paired temporal sampling, VLM recognition, transition voting and macro/micro aggregation.	`VideoTaskRecord` v2 JSON.
Augmentation (B)	`src/semantic_motion/augmentation.py`	Produces source, template or LLM instruction variants with source-step and prompt provenance.	Augmented instructions embedded in task segments.
Preparation (C1)	`src/module_c/prepare_samples.py`	Converts task records to standardized samples and loads real motion tracks.	Per-sample JSONL and pretty JSON.
Refinement (C2)	`src/module_c/run_refinement.py`	Checks alignment, scores motion, verifies semantics and emits explanations.	Per-sample refined JSONL and pretty JSON.
Orchestration	`app.py`, `dashboard/src/App.jsx`	Runs selected stages asynchronously, reports progress and visualizes refined records.	Task-status API and Web dashboard.

Table 3.2. Module responsibilities and exclusions.

Module	Responsible for	Explicitly not responsible for
Rendering	Visual representation of raw motion, camera placement, simple scene context, video encoding and trajectory export.	Semantic labels, motion acceptance decisions or robot control.
Perception	Visual grounding, candidate boundaries, macro intent, micro-actions and evidence references.	Guaranteeing linguistic factuality or physical trajectory quality.
Augmentation	Instruction-style diversity, source-step linkage and rewrite provenance.	Changing observed actions or certifying final sample acceptance.
Refinement	Sample normalization, synchronization diagnostics, physical scoring, semantic verification, decisions and reason codes.	Repairing corrupted source motion or retraining upstream models.
Web layer	Task orchestration, polling, log exposure and visualization.	Changing model outputs, thresholds or scientific metrics.

Table 3.3. Critical cross-module fields.

Contract	Required core fields	Purpose
`ViewBundle`	`sample_id`, `timebase`, `views`, `trajectory`, `provenance`	Associates synchronized visual and physical evidence.
`TaskSegment`	frame/second interval, task text, macro intent, micro-instructions, confidence, evidence by view	Represents one temporally grounded semantic task.
`Sample`	`sample_id`, video/views, text, motion tracks, interval, provenance	Provides the normalized unit consumed by Module C.
`RefinementResult`	motion score, semantic label/confidence, decision, reason codes, auxiliary data	Supports acceptance, audit and visualization.

each view and ordered fixed-then-ego for every timestamp. This ordering is stated in the VLM prompt so that transition indices refer to timestamps rather than individual images.

It is important to distinguish diagnosis from enforcement. At perception time, `validate_view_bundle` returns stable synchronization reasons, but `run_view_bundle` stores them with `validation_enforced=false`. Module C later probes the files again and adds missing-view, unreadable-view, FPS, frame-count and duration reason codes.

3.5.2 Shared Timebase and Motion Trajectory Contract

`SharedTimebase` contains FPS, frame count, duration in seconds and a `FrameMap` relating encoded frames to source frames. A `MotionReference` contains the trajectory path, source, spatial unit, coordinate frame and track names. The application-level orchestrator further requires the Module D convention `data/module_d_output/<sample_id>/<sample_id>_trajectory.json`; it rejects unexpected or escaping paths before starting model inference.

Module C converts supported Module D, LIBERO or standard JSON trajectories into named `MotionTrack` arrays containing positions and timestamps. Formal checks require increasing timestamps, supported units, real rather than dummy motion, and coverage of the sample interval. Track positions are normalized to metres before velocity, acceleration, jerk and jitter indicators are computed. The default configuration uses minimum aggregation across tracks, so the weakest configured track determines the aggregate motion score.

Figure 3.2 illustrates the paired multi-view contract: all views and the trajectory are expressed against one shared timebase and sampled at identical source-frame indices.

3.6 Observability and Visualisation Design

VLM and semantic-verifier calls can take substantially longer than an ordinary HTTP request. The FastAPI service therefore returns a UUID task identifier from `POST /run-pipeline` and executes

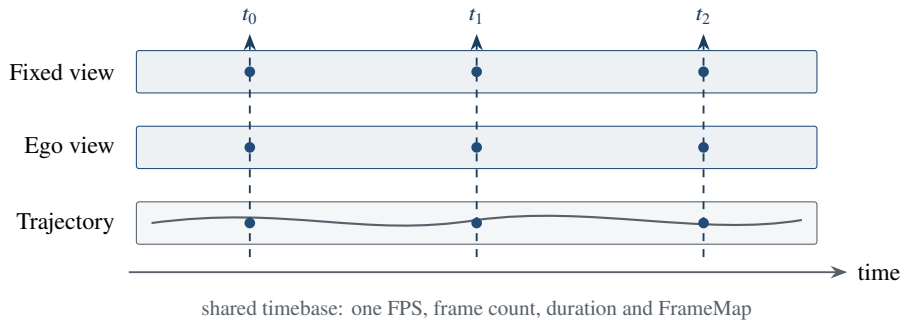

Figure 3.2. Paired multi-view video contract. A `ViewBundle` binds one `sample_id`, a `SharedTimebase`, the synchronized fixed and ego `ViewStreams`, a `MotionReference` trajectory and provenance. Identical source-frame indices (t_0, t_1, t_2) are sampled from every view so that visual and physical evidence at a timestamp refer to the same instant.

the blocking command sequence as a background task. A lock-protected in-memory dictionary stores `queued`, `running`, `completed` or `failed` state, percentage progress, current step, selected sample, captured `stdout/stderr` and any error.

The React client polls `GET /get-status/{task_id}` every 1.5 seconds. It displays a progress bar, current stage and log count, then retrieves `GET /get-results` after completion. Results are read from independent `*_refined.pretty.json` files; the API combines records in memory for display but does not overwrite them as one evaluation artifact.

Figure 3.3 shows the asynchronous request/response sequence between the client, the API and the background pipeline.

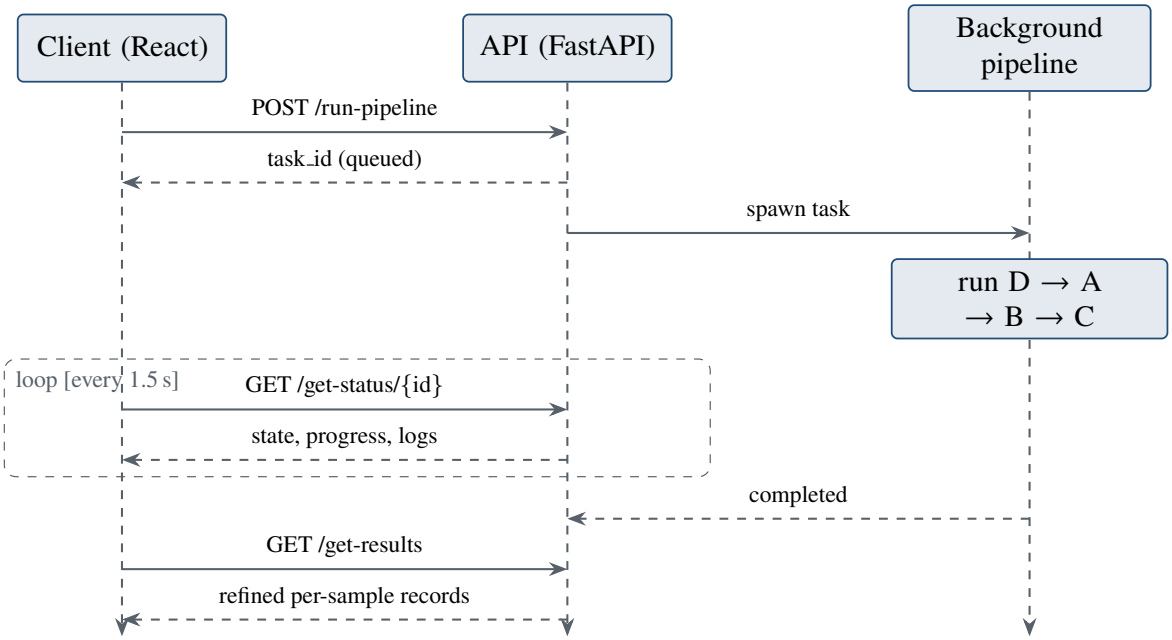

Figure 3.3. End-to-end orchestration sequence. `POST /run-pipeline` returns a task identifier immediately and the pipeline runs as a background task; the client polls `GET /get-status/{id}` every 1.5 s for progress and logs, then retrieves the independent refined records with `GET /get-results`. Solid arrows are calls; dashed arrows are responses.

The dashboard presents processed count, keep rate, mean Motion Quality and Semantic Consistency rate. Motion scores are mapped from $[0, 1]$ to a 0–100 display scale and coloured red for 0–35, amber for 36–59 and green for 60–100. These colours are visual categories only and do not redefine the

YAML threshold. The table retains the backend semantic label, confidence, decision and all reason codes.

This implementation is suitable for local single-process operation. Task state is lost when the server restarts and is not shared across workers. Durable deployment would require persistent task storage, authentication, restricted CORS configuration and a dedicated job queue.

3.7 End-to-End Workflow

The automated formal path is:

1. Module D renders each source motion into fixed and ego videos and exports Root/Hand trajectories.
2. `prepare_module_d_manifest.py` validates each directory and writes one manifest entry per valid sample.
3. The orchestrator reads the manifest, optionally selects one `target_sample_id`, validates its trajectory convention and creates independent output paths.
4. `run_video_task.py` loads the `ViewBundle`. It creates overlapping 16-second macro windows with 8-second stride and 16 sampled timestamps by default.
5. The VLM proposes macro semantics and local transition indices. Hanning-weighted votes from overlapping windows are clustered into global cuts, subject to minimum segment duration.
6. Within each segment, dense 2-second micro windows with 1-second stride and four sampled timestamps produce ordered action primitives. The segment aggregates macro intent, micro-instructions, confidence, evidence and trajectory linkage.
7. Module B produces the configured number of source/template/LLM variants. Current LLM variants carry prompt and source-step provenance but are explicitly marked as not deterministically validated.
8. `prepare_samples.py` selects the configured segment- or video-level text, attaches the sample's real trajectory and writes independent JSONL/pretty JSON.
9. `run_refinement.py` checks cross-modal metadata, scores motion, invokes multi-view semantic verification and writes an independent refined record with explanations.
10. The task status is marked complete and the dashboard refreshes the available per-sample results.

The filename pattern is deliberately stable: `<sample_id>.json`, `<sample_id>_samples.jsonl` and `<sample_id>_refined.jsonl`, each with a corresponding pretty JSON where applicable. This preserves the semantic lineage from one manifest entry to one set of auditable downstream artifacts.

Chapter 4

Design and Implementation of the Four Modules

4.1 The Rendering Module

In the end-to-end data flow of the Semantic-Motion system, the Rendering Module serves as a critical bridge connecting raw motion data with Vision-Language Models (VLMs). Traditional Motion Capture data (such as FBX or BVH formats) essentially consists of a series of 3D coordinate point clouds and skeletal rotation matrices that change over time. Because general-purpose VLMs lack the capability to directly parse pure 3D geometric data, a significant ”representation gap” exists. The design objective of this module is to construct a fully automated, high-fidelity rendering pipeline that transforms abstract ”digital signals” into VLM-perceptible ”visual signals” (multi-view videos), while simultaneously exporting standardized 3D physical trajectories to provide high-quality data sources for downstream semantic perception and quality refinement. The core implementation of this module is built upon the Blender 5.1 Python API.

Figure 4.1 summarizes the rendering pipeline from a raw motion file to its synchronized videos and exported trajectory.

4.1.1 Motion Data Import and Scene Initialisation

To achieve zero-human-intervention processing for large-scale datasets, the rendering module first establishes a robust batch import and scene initialization mechanism. Upon pipeline initiation, the system iterates through the FBX or BVH motion files in the input directory, importing the armature and animation data via Blender’s native interfaces. Addressing the issue of variable action lengths, the script automatically reads the `animation_data.action.frame_range` attribute of the armature to dynamically set the start and end frames of the current scene, thereby achieving adaptive animation durations.

In industrial-grade batch processing, memory leaks and scene pollution are common issues. To address this, the module implements a whitelist-based Trace-free Cleanup Mechanism. Before importing each new action, the system automatically traverses the current scene, destroying the temporary meshes and skeletal data generated in the previous iteration, while strictly preserving foundational scene objects such as lights, cameras, and the ground plane. This state isolation strategy avoids the overhead of repeatedly creating the base environment, significantly enhancing throughput. Furthermore, considering that external motion data may be corrupted, the import phase encapsulates a Try-Except fault-tolerance mechanism. When encountering fatal errors such as an ”invalid header,” the system automatically logs the error and skips the file, ensuring that the entire engineering pipeline does not crash due to a single corrupted data point.

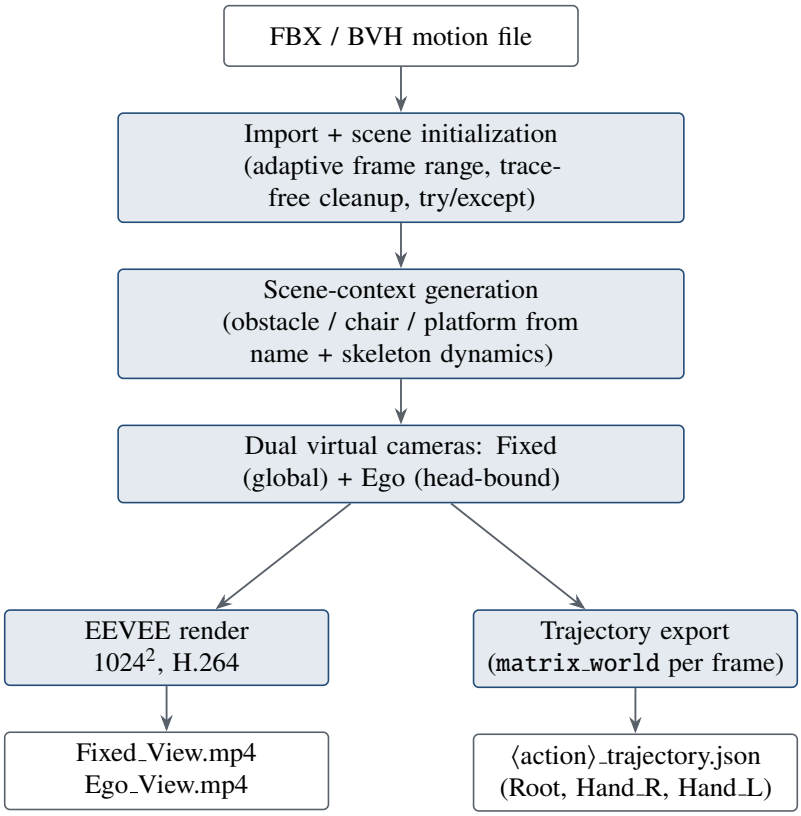

Figure 4.1. Module D rendering pipeline. A single Blender pass imports the motion, injects minimal scene context, and renders synchronized fixed and egocentric videos while exporting a frame-wise 3D trajectory. The video branch feeds Perception; the trajectory branch feeds Refinement.

4.1.2 Scene Context Generation

To assist the VLM in focusing on the action itself and accurately recognizing macro-intents, the module incorporates a dynamic Scene Context Generation algorithm based on semantic parsing and skeletal dynamic tracking. Pure actions (e.g., sitting down in mid-air) are prone to trigger VLM hallucinations or misjudgments. Therefore, by parsing keywords in the motion filenames and combining them with the skeletal coordinates in physical space, the system dynamically injects minimalist geometric anchors:

Vault/Step: When a vaulting or stepping semantic is identified, the system iterates through every frame of the animation to extract the highest point (peak of flight) of the Root/Hips bone along the Z-axis in the world coordinate system. Subsequently, the corresponding (X, Y) coordinates of that frame are projected onto the ground, and a rectangular cuboid obstacle is generated at this geometric center. This ensures that regardless of where the character initiates the jump, the obstacle perfectly aligns with its motion parabola.

Sit: The system shifts to tracking the lowest point of the Hips bone along the Z-axis and generates a cubic chair directly beneath it, resolving the clipping issue where the character appears to "sit in mid-air" during linear displacement.

Drop/Jump Down: For waterfall-type trajectories where the character drops downwards, the system reads the posture of the initial jump frame, iterates through bones such as the Ankle and Toe, and precisely locates the absolute lowest point of the sole. Simultaneously, the algorithm calculates the directional vector from the jump to the landing and applies a dynamic offset to the generated 2x2 meter platform in the opposite direction of the jump. This positions the character exactly at the edge of the platform in the initial frame, perfectly recreating the physical logic of "jumping down from a height."

4.1.3 Multi-view Video Synthesis

To meet the Semantic-Motion system's requirements for multi-granular semantic extraction, the rendering module deploys a dual-track virtual camera system within the 3D engine to simulate a real-world, integrated hardware-software capture environment:

Fixed View: Deployed globally within the scene, this view is utilized to capture the character's full-body kinematics, movement trajectories, and macro-spatial relationships with the environmental context.

Ego View: Aimed at simulating the robot's head-mounted perception. The script uses Python to traverse and lock onto key bones containing "Head" in their names, dynamically binding the camera to the character's head via the `parent_bone` attribute. Subsequently, the viewing direction is corrected using Euler Rotation, enabling it to track the character's line of sight with zero distance. This perspective is crucial for the downstream Perception Module to capture micro-action details, such as Hand-Object Interactions (HOI).

On the rendering output level, the system invokes the Blender EEVEE rendering engine to automatically overwrite the environment background with a solid color (mid-gray/black), eliminating ambient light noise. The output format is configured as a 1024×1024 resolution H.264 encoded MP4 video, ensuring that the VLM is provided with high-definition, interference-free visual inputs even under lower VRAM constraints.

4.1.4 3D Motion Trajectory Export

While generating visual signals, the rendering module is also tasked with extracting physical Ground Truths. Pure video data cannot directly provide precise physical metrics (e.g., jump height, acceleration), which are indispensable for validating motion quality.

This section embeds a trajectory export algorithm within the rendering loop. The algorithm forces an update of the view layer via `bpy.context.view_layer.update()` and reads the product of specific Pose Bones and the `matrix_world` frame-by-frame to obtain absolute physical coordinates. To ensure compatibility with heterogeneous motion capture standards (e.g., Mixamo, CMU), the system features built-in skeletal mapping dictionaries that automatically match and extract the spatial coordinates of key nodes like the Root (center of mass), Hand_R, and Hand_L. The data is truncated to four decimal places to reduce storage redundancy and serialized into JSON format. This design bifurcates the pipeline: the rendered videos flow into the Perception Module, while the precise 3D trajectory data flows directly into the downstream Refinement Module, providing numerical bases for the final Motion Quality Scoring.

4.1.5 Rendering Outputs and Manifest

Following the automated processing of the aforementioned engineering pipeline, the Rendering Module stably generates three core deliverables in the output directory for every inputted raw motion file (e.g., `drop_from_roof.fbx`):

- **Fixed_View.mp4**: A third-person global video featuring the scene context.
- **Ego_View.mp4**: A first-person detail video closely following the character's head movements.
- `<action_name>_trajectory.json`: A 3D motion trajectory file containing the frame-by-frame (X, Y, Z) coordinates of core bones.

These standardized outputs completely eliminate the representation barriers of the raw data, laying a solid underlying data foundation for the system's subsequent semantic generation and multimodal data refinement.

4.2 The Perception Module

The Perception Module is the semantic core of the pipeline. It consumes the synchronized visual evidence produced upstream and returns a versioned `VideoTaskRecord` that a general-purpose VLM cannot produce directly: a temporally segmented, multi-granularity description in which every claim is tied to specific views, timestamps and, where available, a physical trajectory interval. The module adopts the overlapping-window segmentation principle that is common to open-source video-to-task tooling [9], but it extends that principle in two engineering directions required by the rest of the pipeline: a paired multi-view input contract, and a two-level macro/micro output schema with explicit provenance. The implementation is centred on `src/semantic_motion/video_task.py` (`VideoTaskPipeline`), with windowing in `src/semantic_motion/windowing.py`, the input contract in `src/semantic_motion/view_bundle.py`, the recognition adapter in `src/semantic_motion/recognition.py` and the structured prompt in `src/prompts.py`. The module treats the recognition backend as a replaceable component behind a narrow protocol, so a different VLM can be substituted without changing the perception logic. Figure 4.2 summarizes the two-level macro/micro stages that this section describes.

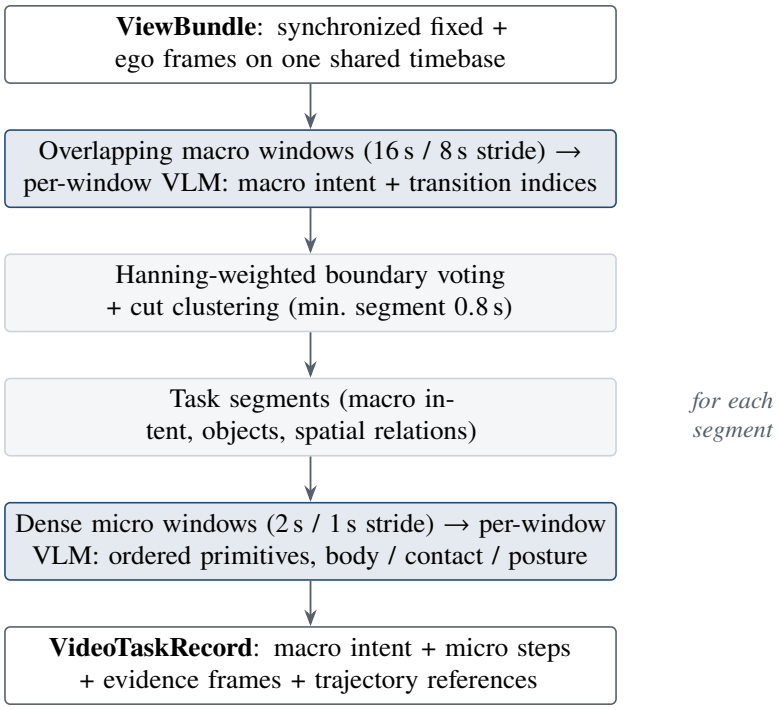

Figure 4.2. Perception Module stages. Overlapping macro windows and weighted boundary voting locate task segments; dense micro windows then describe the ordered action primitives within each segment, yielding a versioned `VideoTaskRecord`.

4.2.1 Paired-view Input and Synchronous Frame Sampling

The formal input is a `ViewBundle` (schema `semantic-motion-view-bundle/v1`) containing a stable `sample_id`, a `SharedTimebase`, a dictionary of named `ViewStream` entries, an optional `MotionReference` trajectory and `Provenance`. Rather than trusting filenames, `build_view_bundle` probes each video with OpenCV to read its true FPS, frame count and duration; the first stream becomes the reference clock and defines a `FrameMap` relating encoded frames to source frames. This guarantees that all views are expressed against one timeline before any inference occurs.

During perception, `extract_bundle_frames` extracts *identical* source-frame indices from every selected view, so the fixed and egocentric images gathered for one timestamp correspond to the same instant of the demonstration. For fused perception, `_ordered_view_paths` interleaves the views by timestamp in a fixed-then-ego order. This ordering is stated explicitly in the VLM prompt so that any transition index the model returns refers to a timestamp rather than to an individual image. The compatibility inputs—a single video or a directory of pre-extracted frames—are wrapped as a `fixed`-only bundle so that the same code path serves both the paired and the debug cases.

4.2.2 Overlapping Macro Windows and Boundary Voting

Task boundaries are proposed from overlapping macro windows rather than from a single pass over the whole clip. `build_temporal_windows` produces windows of 16 s with an 8 s stride and samples 16 evenly spaced timestamps per window by default. Because consecutive windows overlap, each candidate boundary is observed several times, which stabilizes the decision against a single noisy inference. Figure 4.3 illustrates how several overlapping windows contribute weighted votes that agree on a single cut.

For every window, the VLM returns local transition indices. These are mapped to global frames

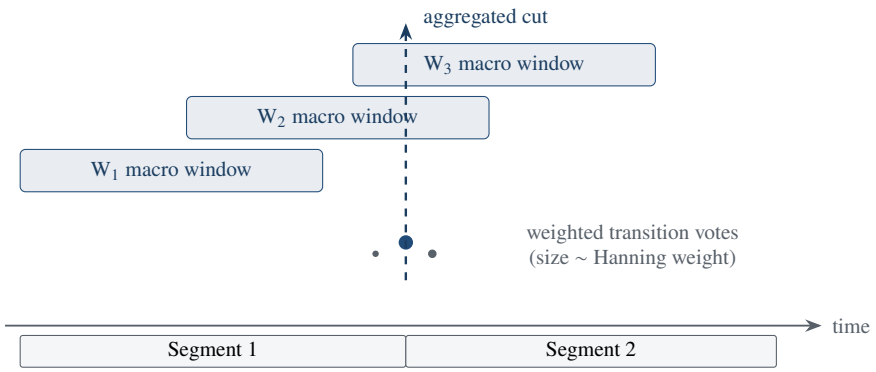

Figure 4.3. Weighted boundary voting. Local transition indices from overlapping macro windows are mapped to global frames and weighted by a Hanning profile (votes near a window centre count more). Clustering the weighted votes yields a single aggregated cut, which partitions the clip into task segments.

and weighted by a Hanning profile (`transition_votes` uses `np.hanning(n+2)[1:-1]`), so a cut proposed near the centre of a window—where temporal context is most complete—contributes more than one proposed at an edge. `aggregate_cut_votes` then clusters the weighted votes within a 2.5 s gap, takes the weight-averaged frame of each cluster as a candidate boundary, and enforces a minimum segment length of 0.8 s both from the previous boundary and to the end of the clip. `intervals_from_cuts` converts the accepted cut frames into half-open frame and second intervals. This weighted-voting scheme is the mechanism that turns many local, uncertain window opinions into a small set of well-supported global boundaries.

4.2.3 VLM-based Structured Scene Understanding

Frame-level understanding is obtained through the `RecognitionModel` protocol. The default adapter, `VLMRecognitionModel`, wraps the repository `VLMEngine` and calls an OpenAI-compatible DashScope endpoint. The system prompt in `src/prompts.py` instructs the model to reason only over visible evidence and to return a strict JSON object with eleven fields: the visible objects, their spatial relations, the task type and domain, an ordered action sequence with per-action details, the target object and destination, a single evidence-grounded summary instruction, the transition indices between atomic tasks, and a confidence value. The exact field names are listed in Appendix A. The model is directed to report `unknown` when evidence is insufficient and not to infer hidden contact or unseen trajectories.

When several images belong to one window, `analyze_many` submits them as one ordered multi-image request. If a recognizer does not support multi-image input, the pipeline falls back to per-image inference followed by a deterministic late fusion (`_fuse_predictions`): scalar fields such as task type and target are resolved by majority vote, list fields are order-preservingly de-duplicated, transition indices are unioned, and confidence is averaged. Because an API response is neither deterministic nor self-validating, the raw response and the model identifier are retained in the record so that later stages can distinguish schema checks from model-mediated judgments.

4.2.4 Macro-Intent Recognition

The coarse intent of each segment is represented by a `MacroIntent` containing `task_type`, `target_object`, `destination`, a `confidence` value and a `domain` restricted to `locomotion`, `manipulation`, `mixed` or `unknown`. Across the overlapping macro windows that cover a segment,

the categorical fields are consolidated by majority vote and the confidence is averaged. When the backend does not expose a confidence, a heuristic estimate (`_prediction_confidence`) is derived from the number of populated structured fields. The result is a single, view-consistent statement of *what* the segment is about, deliberately kept separate from the ordered primitives that describe *how* it is performed.

4.2.5 Dense Micro-Instruction Parsing

Within each accepted macro segment, `build_micro_windows` constructs a dense set of short windows—2 s long with a 1 s stride and four sampled timestamps by default, configurable to a 1–3 s range—so that brief action primitives are not averaged away by the longer macro windows. Each micro window is perceived independently. The ordered primitives of the canonical (first) micro window become the segment's `MicroInstruction` steps, each carrying a `step_id`, natural-language `text`, a `verb/object` decomposition and a confidence. The alternative primitive lists observed in the remaining micro windows are not merged into the step list; they are preserved in segment metadata (`alternative_micro_observations`) as auditable evidence, which avoids double-counting the same action while still recording disagreement between windows.

4.2.6 Body, Contact, Posture and Trajectory Evidence

Beyond verbs and objects, the schema captures the physical grounding needed by the downstream refinement stage. Each entry of `action_details` may carry a `body_part`, a `contact_state` drawn from {`none`, `approach`, `contact`, `grasp`, `release`, `unknown`}, a `posture`, and the image indices that support it. Every `MicroInstruction` additionally stores `evidence_frame_ids`—the source frames that justify the step—and, when a trajectory is attached to the bundle, an `ObservedTrajectoryRef` recording the relevant track names, the step's [start, end] time window and the coordinate frame. This trajectory reference is the explicit link that later allows Module C to score the physical motion that occurred under a described step rather than over the whole clip.

4.2.7 Temporal Task Boundary Detection

Two boundary mechanisms coexist behind the same `TaskSegment` schema. The primary paired path uses the Hanning-weighted vote clustering described above and is the mechanism exercised by manifest and multi-view input. The compatibility single-video path (`run_frames`) instead uses a `TemporalTaskAggregator`, which scans per-frame annotations and marks a boundary when the task type changes or when the content tokens of the target object diverge below a Jaccard similarity threshold (0.30), with short groups merged into their neighbour. Keeping both mechanisms behind one schema means the rest of the pipeline is agnostic to which boundary detector produced a segment.

4.2.8 Segment-level Temporal Aggregation

Each interval is turned into a `TaskSegment` by `_build_windowed_segment`. Objects and spatial relations are de-duplicated across the segment's windows; the task instruction is chosen by majority vote over window instructions, with a deterministic fallback built from the macro intent and micro steps. The segment records its start/end frame and second, an `evidence_by_view` map of the source frames used per camera, the augmented instructions produced by Module B, and diagnostic metadata such as the micro-window count, view mode and whether a trajectory was linked. At the record level,

`VideoTaskRecord` metadata stores the method identifier, window parameters, cut frames, provenance, the generation model, the rewriter identity, and—critically—the synchronization diagnostics together with `validation_enforced=false`. As in the rest of the system, diagnosis is recorded but is not silently used to drop data at this stage.

4.2.9 Static Perception Benchmark

The same structured schema is exercised without the temporal layer through a static single-image benchmark of 30 synthetic scenarios (`run_eval.py`, `SemanticMotionPipeline`). This isolates the model's scene-understanding ability—object recognition, task classification, action ordering, semantic similarity and spatial reasoning—from the additional difficulty of windowing and boundary voting, and it confirms that the JSON contract can represent broad task intent. The quantitative results of this benchmark are reported and discussed in the Perception Module Evaluation of Chapter 5.

4.3 The Fact-preserving LLM Augmentation Module

The Augmentation Module increases the linguistic diversity of the instructions attached to each task segment without changing the observed actions. Its purpose is to expose downstream learning to varied phrasings of the same grounded task, while keeping every variant traceable to the perception steps that justify it. The module is implemented in `src/semantic_motion/augmentation.py` and runs in process as part of `VideoTaskPipeline`, so that augmented instructions are embedded directly inside the task segments rather than produced as a disconnected text file.

4.3.1 Input and Interface

The module operates on a `PerceptionAnnotation`, or on the aggregated annotation of a task segment, and returns a list of `AugmentedInstruction` objects. All rewriters implement a single narrow protocol, `InstructionRewriter.rewrite(annotation, num_variants)`, and are driven through an `AugmentationStream` facade whose default is the conservative source-only rewriter. Because the interface is narrow, template, LLM and source strategies are interchangeable and the pipeline can request a configurable number of variants per segment (`--variants`). This design keeps the augmentation policy replaceable and testable in isolation from perception.

4.3.2 Instruction Reconstruction

Before any rewriting, the module reconstructs deterministic scaffolds from the structured annotation. `_join_steps` composes the ordered micro instructions into a readable plan of the form "*approach the mug, then grasp the mug, and finally place the mug*", and `_intent_phrase` composes a goal phrase from the macro intent of the form "*task target to destination*". These scaffolds serve two roles: they are the raw material for the deterministic template strategy, and they are provided as grounding context to the LLM strategy so that generation is anchored to the perceived steps rather than to free association.

4.3.3 Augmentation Strategies

Three strategies are provided. The **source** strategy (`SourceInstructionRewriter`) is a no-op that preserves the original text and marks it as not augmented; it is the safe default. The **template** strategy (`TemplateInstructionRewriter`) is a deterministic, debug-only rewriter that recombines existing slots into paraphrase, step-expansion and goal-conditioned variants. The **LLM** strategy (`LLMInstructionRewriter`) calls the OpenAI-compatible endpoint using a configurable system prompt (`src/semantic_motion/llm_augmentation_prompt.txt`, overridable through `AUGMENTATION_PROMPT_FILE`), requests the desired number of distinct variants as JSON, de-duplicates them by normalized text, and attaches a strategy label and source-step references to each. The active strategy is selected at run time through `--rewriter none|template|llm`.

4.3.4 Deterministic Fact Preservation and Hallucination Control

Fact preservation is approached through two complementary guarantees with deliberately different strengths, and the report states the current limitation explicitly rather than treating the design objective as fully achieved. The deterministic template strategy *cannot* introduce unseen facts, because it only recombines slots that already exist in the perception annotation; it is therefore the zero-hallucination fallback and is the mode relied upon when strict factual safety is required. The API-backed LLM strategy is more expressive but is not deterministically constrained: it relies on prompt instructions and records `validation=disabled` in its metadata, because the code-level lexical and fact-preservation gates are intentionally not enforced in the current implementation. A deterministic post-generation fact gate for the LLM path remains an acknowledged acceptance gap (FR5). Consequently, factual verification of LLM-generated text is not claimed at this stage; it is deferred to the independent multi-view semantic verification performed by Module C, so that hallucinated variants are caught by cross-modal evidence rather than by an unverified self-check.

4.3.5 Output and Provenance

Every variant is emitted as an `AugmentedInstruction` carrying its `text`, its `strategy`, the `source_step_ids` it derives from, and a metadata block recording the rewriter identity, model, prompt version, prompt file, validation status and, for the LLM path, the raw model response. These variants are stored inside the owning `TaskSegment.augmented_instructions`, an audit trail is written to the segment metadata, and the rewriter identity and prompt version are recorded at the `VideoTaskRecord` level. The result is that each generated phrasing can be traced back to the exact source steps, model and prompt that produced it, which is a prerequisite for any later human audit of augmented training text.

4.4 The Refinement Module

The Refinement Module is the final quality-control layer in the Semantic-Motion pipeline. The front modules render simulation data into videos, infer structured task text based on videos and generate augmented text. However, the generated description may contain an incorrect object, action, temporal order or target state. Meanwhile, the motion trajectory may contain spikes, jitter, invalid timestamps or insufficient observation. These samples are not suitable for downstream VLA training. The role of the refinement module is to ensure that only reliable samples are kept for downstream learning.

The refinement module filters samples containing video, motion and text through two perspectives, namely numerical analysis of motion quality and VLM-based verification of semantic consistency. The outputs of two channels are combined into an explainable keep or drop decision.

Figure 4.4 summarizes this dual-channel decision and the conditions under which a sample is kept.

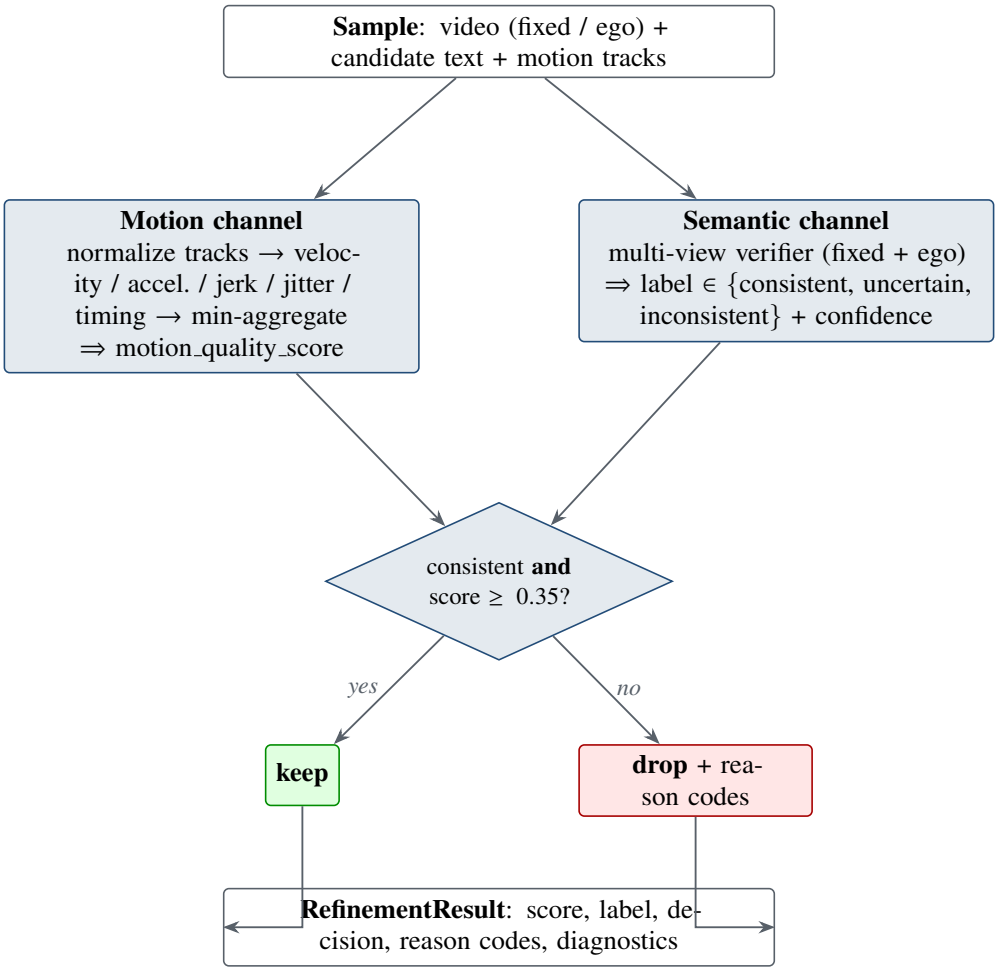

Figure 4.4. Fail-closed dual-channel refinement. A physical motion-quality score and an independent multi-view semantic label are combined conservatively: a sample is kept only when the text is judged *consistent* and the motion score meets the 0.35 threshold, and is otherwise dropped with explanatory reason codes.

4.4.1 Sample Construction

The module first converts the task texts with corresponding videos produced the Perception and Augmentation Modules into a common JSONL representation. Each sample contains a unique sample id, a collection of one or more video paths indexed by views, the texts to be verified, and optional motion tracks.

This intermediate representation allows the refinement process independent from the internal structure of the earlier modules and enables the same procedure to operate on demonstrations of single view, rendering videos of multiple views, and externally prepared datasets.

Two sample granularities are supported. At segment level, every detected task segment becomes an independent sample. Its identifier combines the source demonstration name and segment identifier, and its motion data are restricted to the segment's start and end times. All synchronized views associated

with that demonstration remain attached to the same sample. This mode is appropriate when deciding keep or drop individual sub actions. At video level, the complete video contents are included in the sample regardless of whether the demonstration has one view or multiple views. This mode is more suitable for checking whether a description correctly summarizes the overall tasks.

The text can be selected from the task instruction or augmented instructions. Empty texts, records without any valid video path, and records without task texts are skipped. The absence of one optional view does not make a sample invalid, as long as at least one usable view remains. If no motion track is contained in the sample, the refinement module can operate in semantic-only mode, determining to keep or drop the sample based solely on the task text and the corresponding video. Samples are written both in line-delimited JSON for machine processing, and optionally in indented JSON for manual inspection. This separation between processing and inspection formats improves batch efficiency and ensures observability.

4.4.2 Motion Data Normalisation

Motion data from heterogeneous systems must be converted to consistent formats before a common quality metric can be applied. The refinement module represents motion as a dictionary of named tracks. Each position is converted to a three-dimensional float vector. A valid track must contain at least four observations, have the same number of positions and timestamps, and use strictly increasing timestamps. Invalid tracks are dropped during preparation.

Four input formats are currently supported. Firstly, LIBERO trajectories provide the position of agent's end effector in each action step. These positions are converted into a continuous end-effector track. If the source includes an FPS value, timestamps are derived from the step time and FPS. Secondly, trajectories generated by the rendering module are composed by the three-dimensional location of each selected body joint in every animation frame. The refinement module sorts the frames in chronological order and converts their frame numbers into times, then constructs separate trajectories for the body root, right hand and left hand. Thirdly, a standard single-trajectory file directly provides an ordered list of three-dimensional positions together with the corresponding observation times. The module treats this list as an unnamed trajectory. Finally, a standard multi-trajectory file provides several named position sequences and their observation times. These trajectories and their original names are retained.

For segment-level samples, every track is temporally cropped to the corresponding segment interval and evaluated again. The loader also supports a plug-in interface for additional motion formats. If no real trajectory can be resolved, the pipeline can generate a short linear placeholder trajectory to keep the engineering workflow executable. This placeholder will not be interpreted as evidence of genuine motion quality. By default, the refinement module only verifies semantic analysis when there is no motion track in the sample.

4.4.3 Motion Quality Scoring

For a valid track with positions $\mathbf{p}_i$ at times t_i, let $\Delta t_i = t_{i+1} - t_i$. The implementation clips each time interval from below by $\varepsilon = 10^{-6}$ for numerical safety and computes discrete velocity, acceleration, and jerk as

$$\mathbf{v}_i = \frac{\mathbf{p}_{i+1} - \mathbf{p}_i}{\max(\Delta t_i, \varepsilon)}, \qquad \mathbf{a}_i = \frac{\mathbf{v}_{i+1} - \mathbf{v}_i}{\max(\Delta t_{i+1}, \varepsilon)}, \qquad \mathbf{j}_i = \frac{\mathbf{a}_{i+1} - \mathbf{a}_i}{\max(\Delta t_{i+2}, \varepsilon)}.$$

For uniformly sampled motion, all intervals equal the constant sampling period Δt, so the three denominators above have the same numerical value. The velocity, acceleration, and jerk abnormality ratios are the proportions of their Euclidean magnitudes that exceed the configured limits:

$$r_v = \frac{1}{N_v}\sum_i \mathbb{I}(\|\mathbf{v}_i\|_2 > v_{\max}), \quad r_a = \frac{1}{N_a}\sum_i \mathbb{I}(\|\mathbf{a}_i\|_2 > a_{\max}), \quad r_j = \frac{1}{N_j}\sum_i \mathbb{I}(\|\mathbf{j}_i\|_2 > j_{\max}).$$

Here, $\mathbb{I}(\cdot)$ is the indicator function. The default limits are $v_{\max} = 2.5$, $a_{\max} = 6.0$, and $j_{\max} = 20.0$. They represent the largest motion magnitudes treated as normal for an individual sample. Exceeding a threshold marks that sample as abnormal, but does not by itself reject the entire track. The limits are configurable and should be adjusted when the expected motion scale or sampling characteristics of a dataset differ.

Jitter is measured from changes in the full three-dimensional velocity direction. For every velocity satisfying $\|\mathbf{v}_i\|_2 > 10^{-8}$, define

$$\mathbf{u}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|_2},$$

and let

$$\mathcal{D} = \left\{ i : \|\mathbf{v}_i\|_2 > 10^{-8} \ \wedge \ \|\mathbf{v}_{i+1}\|_2 > 10^{-8} \right\}.$$

The jitter ratio is the proportion of valid adjacent velocity pairs whose directions differ by more than 90°:

$$r_d = \begin{cases} \dfrac{1}{|\mathcal{D}|}\displaystyle\sum_{i\in\mathcal{D}} \mathbb{I}\left(\mathbf{u}_i^\top \mathbf{u}_{i+1} < 0\right), & |\mathcal{D}| > 0, \\ 0, & |\mathcal{D}| = 0. \end{cases}$$

Normalization $\mathbf{v}_i$ removes the influence of magnitude, so this measure responds only to changes in direction. Velocities below 10^{-8} are excluded because their directions are undefined or numerically unstable. Since both $\mathbf{u}_i$ and $\mathbf{u}_{i+1}$ are unit vectors, their dot product equals the cosine of the angle θ_i between them:

$$\mathbf{u}_i^\top \mathbf{u}_{i+1} = \cos\theta_i.$$

A negative dot product therefore indicates $\theta_i > 90°$, which the implementation counts as a rapid direction reversal. Consequently, for $r_d \in [0, 1]$, a value near zero indicates few such reversals, while a larger value indicates more frequent directional oscillations. If there is no valid adjacent velocity pair, r_d is defined as zero.

Two additional measures detect timing anomalies. The interval coefficient of variation c_t and the largest-gap ratio g_t are

$$c_t = \frac{\mathrm{std}(\Delta t)}{\max(\mathrm{mean}(\Delta t), \varepsilon)}, \qquad g_t = \frac{\max_i \Delta t_i}{\max(\mathrm{median}(\Delta t), \varepsilon)}.$$

The coefficient c_t measures the overall relative variation of the sampling intervals. It is zero when all intervals are equal and increases as the sampling schedule becomes less regular. The ratio g_t

compares the largest interval with the median, which represents the typical frame interval and is less sensitive to isolated gaps than the mean. Thus, $g_t = 1$ for uniform sampling, while a significantly larger value indicates an unusually long interval that may result from a dropped frame or a timestamp shift. The terms ε only prevent division by a reference interval near zero.

The implementation therefore forms six normalized penalties. The first three use a ratio of abnormal samples of 0.2; the other three use their configured limits $r_{d,\max} = 0.30$, $c_{t,\max} = 0.25$, and $g_{t,\max} = 2.5$:

$$q_v = \min(1, r_v/0.2), \qquad q_a = \min(1, r_a/0.2), \qquad q_j = \min(1, r_j/0.2), \qquad q_d = \min(1, r_d/r_{d,\max}),$$

$$q_c = \min(1, c_t/c_{t,\max}), \qquad q_g = \min\left(1, \frac{\max(0, g_t - 1)}{\max(g_{t,\max} - 1, \varepsilon)}\right).$$

The quality score for one track is

$$s_{\text{track}} = \text{clip}\left(1 - \frac{q_v + q_a + q_j + q_d + q_c + q_g}{6}, 0, 1\right).$$

Consequently, a smooth trajectory with regular sampling receives a score near one, whereas frequent kinematic spikes, direction reversals, irregular intervals, or large time gaps reduce the score. The corresponding reason codes are `high_velocity_spikes`, `high_acceleration_spikes`, `high_jerk_spikes`, `high_jitter`, `irregular_frame_intervals`, and `drop_frame_or_time_shift`. Tracks with fewer than four samples, non-finite values, or non-increasing timestamps receive a score of zero and a corresponding validation reason.

When multiple tracks are present, their scores are aggregated using either the minimum or the mean. The default is the minimum, so a single track with a poor rating is sufficient to drag down the complete sample's motion score. This is appropriate for strict data cleaning because a smooth root trajectory should not conceal an unstable hand trajectory. Mean aggregation is available when overall motion quality is more important than the weakest component.

4.4.4 Semantic Consistency Verification

Motion smoothness alone cannot determine whether a text describes the visual task correctly. The semantic channel submits all available views, together with the candidate description, to the multimodal verifier. A LIBERO sample is verified from its video of single agent view and eye in agent's hand. A rendering module sample is verified jointly from the egocentric and fixed third-person views. The first-person view provides evidence of subject interaction, while the fixed view provides motions of full body and scene context. By treating these two videos as complementary observations of the same sample, the system can avoid making conflicting decisions for each camera.

The verifier uses qwen3-vl-plus through an endpoint compatible for OpenAI. For each available view, a local video of no more than 18 MB can be encoded and sent directly. For a larger video, the system uniformly extracts eight temporally ordered JPEG frames at quality 85. The view name is retained when constructing the request so that evidence from the egocentric and fixed cameras can be distinguished.

The verification prompt evaluates seven dimensions: actor correctness, object correctness, action correctness, temporal correctness, goal and final-state correctness, hallucination, and evidence

sufficiency. The verifier makes decisions relying only on visible evidence. Ambiguous evidence produces an "uncertain" result, rather than an unsupported guess.

The normalized output contains a label, which may be "consistent", "uncertain" or "inconsistent", a confidence value between zero and one, a list of error types, and a suggested replacement text. Confidence represents confidence in the assigned label. For example, "inconsistent" with high confidence indicates strong evidence of a mismatch. Error tags identify failures such as object_mismatch, action_mismatch, possible_hallucination, and insufficient_evidence.

4.4.5 Deterministic Cross-modal Alignment Checks

The final decision combines two quality channels conservatively. A sample is retained only when its semantic label is "consistent" and the motion score meets or exceeds the default threshold of 0.35. If no motion score is available, semantic consistency alone determines whether the sample can be retained.

A sample with correct semantical descriptions is dropped when its motion score is below the threshold. A smooth trajectory is also dropped when its descriptions are uncertain or inconsistent. When motion is absent, the sample enters semantic-only mode and may still be kept if its semantic label is "consistent". This procedure ensures that the entire process is operational while the condition that motion was not evaluated is explicitly recorded.

4.4.6 Fail-closed Refinement Decision and Explainable Outputs

Each result of refinement module records the sample identifier, motion-quality score, semantic label, semantic confidence, final decision, and a list of reason codes. It also retains the scores and abnormality ratios of per track, semantic error types, suggested text, and original candidate text. Results are saved as JSONL for downstream processing and as formatted JSONL for manual review. This output design makes rejection traceable. A drop decision may be attributed to a threshold failure such as "low_motion_score", to an issue of a specific track such as "Hand_R: high_jitter", or to a semantic problem such as "object_mismatch".

The Refinement Module closes the engineering loop of four modules. The Rendering Module supplies visual and motion signals. The Perception Module extracts structured semantics. The Augmentation Module generates text variants. The Refinement Module determines whether the aligned sample is sufficiently reliable.

Chapter 5

System Integration, Demonstration and Evaluation

This chapter evaluates Semantic-Motion Engine as an integrated engineering system, rather than just four isolated models. The four stages follow the actual data flow. First, the Rending Module converts motion or simulation data into videos of first-person and third-person view. Second, the Perception Module extracts structured task semantics from related videos. Third, the Augmentation Module generates command variants. Fourthly, the Refinement Module evaluates the motion quality and semantic consistency of samples to determine whether to filter of retain them. The evaluation verifies the operation of the individual modules and the transfer of videos, motions and text between consecutive stages.

5.1 Integration Environment and Technology Stack

The integrated software pipeline was implemented in Python 3.10 or later. OpenCV is used for video decoding, frame sampling, and local video preprocessing. Structured data are defined as Python classes with fields and data types, and are exchanged between modules via JSON or JSONL files. YAML configuration files store motion thresholds, synchronization requirements, settings of semantic verifier, and model parameters.

The Rendering Module uses Blender to convert motion files into videos of egocentric and fixed view while exporting three-dimensional joint trajectories. Visual-language inference is implemented through a DashScope API compatible with OpenAI. API credentials, endpoint addresses, and model identifiers are supplied through environment variables rather than source files. This avoids embedding secrets in the repository and allows to change inference models without modifying project codes.

5.2 Module Integration and Interface Verification

The principle integration object is a view bundle. It associates a sample identifier with a shared time base, fixed and egocentric videos, a trajectory source, and source information. The views in a valid formal multi-view sample share the same frame rate, frame count, and duration, allowing observations from different cameras to be compared at corresponding times.

The Perception Module processes the view bundle and produces a versioned video-task-record. This record contains corresponding video path, sampled multi-view frames, temporal task segments, macro-intents, micro-instructions, detected objects and spatial relations. The Augmentation Module receives the structured segment description, and every generated variant retains its relationship between source steps.

Before refinement, the perception record is converted into a uniform sample, containing candidate texts, video views, segment boundaries and motion tracks. LIBERO's positions of end effectors and Rendering Module's trajectories of body joints are normalized into the track representation with the

same name. This interface allows the motion scorer to operate without knowing the original dataset format.

Interface verification was performed on the LIBERO sample "put_the_bowl_on_the_plate", which contains views of agent and eye in hand. Both views contain 90 frames at 20 frames per second and have a duration of 4.5 seconds. The synchronization check reports a valid sample of paired views. The first generated segment covers 0.00–3.35 seconds, and its end-effector trajectory covers the same interval. The second segment covers the remainder of the demonstration. The provenance records the dataset, source identifier, generator, and source revision. These checks show that visual, textual, temporal, and motion data remain associated after passing through multiple modules.

Schema validation prevents several engineering failures. For example, a view cannot be detached from its trajectory without detection and motions in segment level cannot be evaluated against unrelated time intervals. Intermediate files in standard format and structured error codes also prove for observability at module boundaries.

5.3 End-to-End Demonstration

The main end-to-end demonstration uses the Module D jump_down motion. The Rendering Module imports the action of motion capture into Blender, identifies it as a downward jump action, and places a platform relative to the character's take-off position and movement direction. It renders Fixed_View.mp4 from the scene camera and Ego_View.mp4 from a camera attached to the head bone. At the same time, it exports jump_down_trajectory.json, which contains the world coordinate positions of Root, Hand_R and Hand_L at each animation frame. The resulting paired videos each contain 70 frames at 30 frames per second and last 2.33 seconds.

The rendered files are indexed by manifest.json as one jump_down view bundle. The Perception Module processes the fixed and egocentric videos jointly in the mode of fused view and associates the exported joint trajectory to the same record. It identifies a cut at frame 35 and produces two temporal segments. The first segment, from 0.00 to 1.17 seconds, is described as the humanoid crouching on a block, jumping upward, reaching with both hands, and catching a floating low-poly head-like object. The second segment, from 1.17 to 2.33 seconds, is described as a transition from standing to a low crouched posture supported by the hands and feet without interaction with other objects. The stored perception record was generated with qwen3-vl-plus.

The Augmentation Module generates three rewritten instructions for each segment, producing six variants in total. These variants retain the associations with the four source microsteps in their respective segments and take strategies such as temporal compression, goal-oriented framing, stage-based decomposition and spatial context enhancement.

Before refinement, prepare_samples combines the two segment descriptions into one candidate in video level while retaining both source segment identifiers, video paths 30 FPS shared time base, and all three motion tracks. The synchronization check confirms that the fixed and egocentric videos both contain 70 frames and have matching 2.33 second durations. The Root, Hand_R and Hand_L tracks cover 0.00–2.30 seconds. The visual and motion evidence are temporally aligned.

The Refinement Module evaluates the three tracks with the aggregation of minimum score. Root and Hand_L each score approximately 0.5000, while Hand_R scores 0.4918. Therefore the aggregate motion-quality score is 0.4918. All three tracks trigger high-velocity, high-acceleration, and high-jerk diagnostic codes, reflecting the rapid take-off and landing dynamics of the jump. Since the motion

threshold is 0.35, the aggregate score under the default configuration still passes the motion gate. Joint multi-view semantic verification labels the combined task text "consistent" with confidence 0.95 and returns no semantic error types.

The recorded refinement decision is "keep". This decision follows the Refinement Module rule: a sample is retained only when the motion-quality score is not below the configured minimum and the semantic label is "consistent". Otherwise it is dropped. In the present case, both conditions are satisfied, so the sample is kept even though the reason code contains spike diagnostic information of track.

5.4 Perception Module Evaluation

The static perception module benchmark was evaluated on 30 scenarios of robot tasks and structured ground truth. It evaluates object recognition, task classification, action sequence similarity, semantic similarity, and spatial reasoning.

Six complementary metrics are used. Object recognition F1 measures set overlap between predicted and ground-truth object names after normalization. Task classification accuracy is a binary score that equals one only when the predicted task type matches the labelled type exactly. ROUGE-L of action sequence compares the longest common subsequence between the predicted and reference action plans and therefore rewards both content and order. Semantic similarity aggregates the predicted objects, task type, actions, and target into a single bag-of-words representation and computes cosine similarity against the corresponding ground-truth text. Spatial reasoning accuracy counts the fraction of labelled spatial relations recovered in the prediction through substring matching. The composite score is the equally weighted average of these five metrics, each contributing a weight of 0.2. The results are shown below.

Metric	Score
Object recognition F1	0.7602
Task classification accuracy	1.0000
Action-sequence ROUGE-L	0.7176
Semantic similarity	0.9273
Spatial reasoning accuracy	0.6278
Composite score	0.8066

Table 5.1. Static Perception Module benchmark results on 30 scenarios using Qwen-VL-Plus.

The model correctly classified the task type in all 30 synthetic scenes and achieved high overall semantic similarity. This indicates that the structured output format can represent broad task intent effectively. Object recognition and action-sequence matching were weaker but remained substantially above zero. Spatial reasoning produced the lowest score. Inspection of predictions shows that support surfaces and appliance subcomponents were often described too generically, causing mismatches even when the overall task was understood.

5.5 Pinned Video2Tasks Full-pipeline Comparison

To position the Perception Module relative to the open-source video-to-task baseline it draws its windowing principle from, the exact upstream Video2Tasks revision is vendored and hash-verified,

and its full segmentation pipeline (overlapping windows, weighted voting and `build_segments_via_cuts`) is executed unchanged [9]. The comparison holds the model, window length, stride, frames per window and token budget constant across two arms, so that only the instruction prompt differs. Filename and closed task-list priors are disabled so that neither arm can exploit dataset-specific shortcuts.

On a small WGO-Bench robot subset with human timestamped subtask labels, the pinned upstream prompt reaches an instruction token F1 of 0.200 against 0.116 for the Semantic-Motion prompt, measured over one annotated episode with three repeats. A larger 100-sample manifest run was also executed as an integration exercise, but only a single sample carried usable temporal gold in the recorded artifact, so its aggregate numbers are reported as a pipeline smoke-test rather than as a benchmark result. Neither artifact establishes superiority of either prompt. Two caveats are essential and are stated to avoid over-interpretation. First, this experiment is a *prompt ablation inside the upstream harness*: the Semantic-Motion arm here is a blind single prompt, not the full multi-view `VideoTaskPipeline` with macro/micro aggregation, so it does not measure the actual Fixed/Ego/Fused behaviour of Module A. That behaviour must be evaluated with `tools/evaluate_semantic_motion.py` against adjudicated annotations. Second, the subset is far too small for statistical claims; the paired bootstrap interval is undefined at this sample size. The comparison is therefore reported as an integration and fairness check, confirming that both prompts run through one pinned aggregation path under identical settings.

5.6 Refinement and Alignment Evaluation

5.6.1 Human Gold Annotation Protocol

Formal refinement metrics require human ground truth that the automated pipeline cannot supply. `tools/create_gold_annotation_packets.py` produces 20–30 paired-view annotation packets whose fields include subtask boundaries, segment labels, semantic slots and a keep/drop judgment. Each packet remains in a `pending_human` state until two independent annotators and an adjudicator complete it, and the metric tooling deliberately rejects pending packets: the current adjudicated pool is empty, so `offline_pending_gold_metrics.json` reports zero formally eligible samples. In consequence, no formal boundary accuracy, keep/drop accuracy or calibration figure is claimed in this report. The subsections below therefore describe the implemented metric machinery and report only the weak-proxy and controlled-injection evidence that does not depend on the pending human gold.

5.6.2 Temporal Boundary and Segment Metrics

The metric suite in `src/evaluation/metrics.py` implements greedy one-to-one boundary matching within a ±0.5/1 s tolerance, segmental F1 at IoU 0.5/0.75, an end-to-end labeled segment F1 that requires both temporal overlap and a label match, per-slot token F1, a normalized order-edit score, and a paired bootstrap confidence interval. As no adjudicated temporal gold yet exists, these metrics are exercised against a deliberately *weak* proxy: selected LIBERO Goal demonstrations in which the official task title is treated as a single full-video segment. Table 5.2 summarizes 27 valid runs (3 samples × 3 views × 3 repeats).

Two observations follow. Boundary F1 is zero for every view because the title-as-one-segment proxy contains no interior boundary, so any temporal cut is scored as over-segmentation; this is a

Table 5.2. LIBERO Goal title-as-single-segment weak-proxy results (Qwen3-VL-Flash, 27/27 valid runs). Scores measure agreement with the title-as-one-segment experimental definition, not human atomic-subtask boundaries.

View	Title token F1	Slot macro F1	Segment F1 @IoU 0.5	Semantic label acc.	Mean latency (s)
Fixed	0.168	0.000	0.667	0.000	40.1
Ego	0.288	0.165	0.667	0.000	37.7
Fused	0.223	0.255	0.667	0.333	75.5

property of the proxy, not necessarily of the predictions. On this proxy, fused input improves the title token F1 over the fixed view by +0.055 (paired bootstrap 95% CI [+0.020, +0.089]) and adds non-zero slot and end-to-end labeled scores, but it trails the ego view by −0.065 ([−0.119, −0.003]). The interpretation is modest and consistent with the static benchmark: for these short manipulation clips the egocentric view already exposes the hand–object interaction that dominates the title, and fusion mainly helps relative to a fixed-only view.

5.6.3 Keep/Drop and Confidence Calibration

The refinement decision machinery is fully implemented: binary classification metrics with a false-keep rate and AUROC, Brier and expected calibration error, and coverage–accuracy curves over confidence thresholds. However, the current policy disables Module C quality gates, so every prepared sample receives `decision="keep"` with a `quality_gates_disabled` reason code, and the semantic channel additionally lacks adjudicated keep/drop labels. Under these conditions keep/drop accuracy and calibration are *diagnostic-only* and are not reported as performance. What can be verified is that the decision *logic* behaves as specified when exercised on concrete records: a sample is retained only when its semantic label is `consistent` and its motion score meets the 0.35 threshold, and it is otherwise dropped, as demonstrated in the case studies below.

5.6.4 Controlled Motion Corruptions

Because human motion-quality labels are unavailable, the motion channel is calibrated with controlled, reproducible corruptions injected into a clean reference trajectory: jitter, spike, dropped frames, time shift and NaN values. Each corruption is produced deterministically (`src/evaluation/corruption.py`) while keeping the trajectory parseable. Table 5.3 reports the detector on the end-effector track of a LIBERO manipulation demonstration.

Table 5.3. Motion-quality scores under controlled single-track corruptions (LIBERO end-effector track). Detection AUROC over clean vs. corrupted is 1.0 on this track; this is a single-track calibration diagnostic, not a benchmark claim.

Corruption	Motion score	Triggered reason codes
Clean	0.990	—
Jitter	0.333	`high_jitter`, `high_velocity_spikes`, `high_acceleration_spikes`, `high_jerk_spikes`
Spike	0.892	— (single mid-track spike diluted at this severity)
Drop frame	0.638	`irregular_frame_intervals`, `drop_frame_or_time_shift`
Time shift	0.638	`irregular_frame_intervals`, `drop_frame_or_time_shift`

The scorer separates clean from corrupted trajectories with perfect ranking on this track, and the reason codes correctly localize the failure type. The spike case is instructive: a single isolated outlier over an otherwise smooth track raises the score only modestly, because the abnormality ratio is averaged over all samples. This is an honest limitation of the current aggregation rather than a bug, and it motivates severity-aware or peak-sensitive penalties as future work.

5.7 Cross-module Case Studies

The value of the pipeline is most visible when the channels disagree. The end-to-end `jump_down` demonstration presented earlier in this chapter already shows the agreement case, where smooth-but-spiky motion passes the gate and consistent semantics lead to `keep`. The complementary disagreement case comes from the LIBERO `put_the_bowl_on_the_plate` sample under the Fixed/Ego/Fused ablation. Here the fused perception produced two segments with a cut at frame 67 and a plausible instruction ("*place the black bowl onto the white plate…*"), and the motion channel scored the trajectory very highly (0.987 and 1.000 for the two segments). Nevertheless, the independent multi-view verifier labelled both segments `inconsistent`, emitting object, action and goal-state mismatch codes across views, and the refinement decision was correctly `drop` for both. This is exactly the failure mode a single-channel filter would miss: a physically smooth trajectory paired with an unfaithful description. The case also carries its own caveats, recorded in the artifact: the ground truth is a weak task title, and the same Qwen model family was used for both generation and judging, which can inflate apparent agreement and must be avoided in any formal evaluation.

5.8 Component and Configuration Analysis

Several configuration choices were examined through the same runs. The **view mode** is the most consequential: on the LIBERO proxy the ego and fused views clearly outperform the fixed view for short manipulation clips, at a latency cost—fused perception averages 75.5 s per run versus 37.7 s (ego) and 40.1 s (fixed), because it issues inference over interleaved two-view evidence, at an estimated cost near $0.001 per run. The **motion aggregation** rule defaults to the minimum across tracks, so the weakest track determines the sample score; this maximizes interception of localized anomalies (high precision) at the risk of discarding otherwise good samples (lower recall), and mean aggregation is available when overall quality matters more than the worst track. The **motion thresholds** ($v_{\max} = 2.5$, $a_{\max} = 6.0$, $j_{\max} = 20.0$, motion floor 0.35) and the **window schedule** (16 s/8 s macro, 2 s/1 s micro) are configuration-driven and should be retuned when the motion scale or frame rate of a dataset differs. Finally, the **augmentation strategy** trades safety for diversity: the deterministic template mode cannot hallucinate but is linguistically narrow, whereas the LLM mode is diverse but currently records `validation=disabled` and relies on downstream semantic verification.

5.9 Engineering Acceptance Summary

Table 5.4 summarizes the status of each requirement against the evidence actually available, distinguishing automatically verified behaviour from API-dependent, Blender-dependent and pending-human evidence, in line with the project's rule of not inferring completion.

Table 5.4. Engineering acceptance status by evidence class.

Requirement	Status	Evidence and caveats
FR2, FR3, FR8, FR9	Automatically verified	Manifest indexing, shared timebase, independent per-sample artifacts and selective/batch execution covered by unit tests.
FR1	Blender-dependent	Rendering demonstrated on sample motions; no new acceptance claim without a reproducible Blender scene.
FR4	Structure verified, content API-dependent	Multi-granularity schema and windowing verified offline; field *content* depends on the VLM.
FR5	Partially met	Provenance and source-step linkage verified; deterministic fact gate for LLM variants is an acknowledged gap.
FR6, FR7	Structure verified, semantics API-dependent	Sync/motion scoring and explainable outputs verified deterministically; the semantic channel depends on the verifier API.
FR10	Design specified	Asynchronous task-status API and polling described in Chapter 3; a durable, authenticated, multi-worker deployment is out of scope here.
NFR1–NFR3	Automatically verified	Traceability fields, file-based modular contracts and structured failure reporting exercised by tests.
NFR4	Partially met	Windowing and scoring are deterministic; external VLM/LLM responses are marked model- and API-dependent.
NFR5	Met	The semantic stages run against a cloud endpoint without local training, with the stated cost/reproducibility trade-off.

Chapter 6

Engineering Discussion

6.1 Interpretation of System Results

From the perspective of end-to-end execution, the system demonstrates exceptional capabilities in ”macro-task understanding,” yet reveals clear performance bottlenecks in ”micro-action and spatial parsing.” The Vision-Language Model (VLM) achieves a high accuracy rate in recognizing macro-intents (e.g., ”moving a box”), indicating that the large model's prior knowledge of general scenes generalizes well to the robotic rendering perspective. However, when processing spatial relationships and fine-grained operations (e.g., determining the exact distance and contact state between the end-effector and the object), the accuracy drops significantly.

Furthermore, the system outputs reveal a strong presence of cascading errors. A perceptual error or hallucination by the VLM in a single frame directly propagates into the temporal boundary detection of the task, leading to over-segmentation or erroneous merging of action segments. This explains why relying solely on vision-based large model predictions is insufficient, and it validates the absolute necessity of the dual-channel refinement mechanism (incorporating both ”motion trajectory quality” and ”semantic consistency”) in the Refinement Module. The dual-channel approach is significantly more effective at intercepting misleading data samples than relying on a single visual metric.

6.2 Engineering Strengths

The greatest engineering value of this project lies in the construction of a runnable, observable, and extensible closed-loop data infrastructure, rather than merely algorithmic fine-tuning:

Lightweight and Low Resource Dependency: The pipeline design drastically lowers the computational threshold. By calling cloud-based VLM APIs for visual understanding tasks, the system can run smoothly without the need for local high-performance GPU clusters.

High Modularity and Multi-source Compatibility: The four modules of the system are highly decoupled. This not only allows for seamless integration of more advanced VLMs or LLM rewriters in the future, but also enables the system to be compatible with highly divergent data sources. Whether processing raw FBX motion capture data with complex skeletal hierarchies or standardized LIBERO robotic trajectories, the system can clean and align them through unified data interfaces.

Strong Interpretability of Decisions: The automated data refinement process is not a black box. The output from the Refinement Module is not just a binary keep/drop decision; it includes detailed motion anomaly penalties (e.g., `high_jerk_spikes`) and semantic mismatch reason codes, providing solid data support for subsequent manual review and system debugging.

6.3 Failure Mode Analysis

During actual engineering execution, the system exposed several typical failure modes:

Visual Feature Confusion and Object Misidentification: The VLM often misidentifies interfering

objects in the background as target objects, or experiences semantic confusion between visually similar components.

Information Loss due to Temporal Sampling: The uniform frame sampling strategy, adopted to balance cost and efficiency, may inadvertently miss critical interaction moments that last only for a fraction of a second, resulting in gaps in the generated micro-instruction descriptions.

Spatial Alignment and Rendering Biases: In the Rendering Module, when simple scene contexts (like obstacles or boxes in a vaulting action) are dynamically generated based on skeletal trajectories, spatial misalignments occasionally occur between the generated obstacle and the actual skeletal movement. This spatial dislocation in the rendered environment directly induces the VLM to produce false ”non-contact” or ”collision” hallucinations.

Boundary Collapse Triggered by Single-frame Hallucinations: Even if the action itself is continuous, if the VLM hallucinates in a single frame (e.g., falsely concluding that the target object has changed), the temporal segmentation algorithm will misjudge it as the start of a new task, leading to a logical fragmentation of the entire task sequence.

6.4 Limitations

Although Semantic-Motion provides a complete end-to-end pipeline, the current system is still constrained by several key factors:

Embodiment Gap: The Rendering Module currently processes human skeletal motion capture data. The degrees of freedom and kinematic characteristics of human motion possess an inherent embodiment gap compared to actual robotic arms (like UR5 or Franka). This implies that high-quality action descriptions refined based on human models may still require further alignment mapping before being directly transferred to robotic action generation models.

Lack of Large-scale Gold-label Data: The evaluation of Module C is currently limited by a small scale of video cases and static benchmarks (predominantly synthetic images). The lack of large-scale, human-expert annotated datasets for real robotic operations makes it difficult to rigorously prove the absolute performance gain of the dual-channel refinement on the final VLA training efficacy from a statistical standpoint.

Uncontrollability of External Dependencies: While heavily relying on cloud APIs brings convenience, it introduces risks regarding reproducibility. Silent updates to the underlying cloud models may result in different semantic outputs when feeding the exact same video input.

Limitations of Text Augmentation: Although the current rule-template-based Augmentation Module guarantees zero-hallucination, it sacrifices the rich diversity of natural language. It cannot generate instruction variants encompassing complex environmental modifiers or tonal shifts like a native LLM.

6.5 Engineering Trade-offs

The core process of building this system is essentially a collection of engineering trade-offs:

Convenience of Cloud APIs vs. Controllability of Local Deployment: Utilizing cloud APIs lowered the initial construction cost and hardware threshold but sacrificed local execution determinism and version-locked reproducibility.

Cost Advantage of Uniform Sampling vs. Temporal Information of Keyframes: Frame-by-

frame analysis carries an exorbitant computational overhead, making uniform sampling a necessary compromise. While it improves batch processing efficiency, it inevitably increases the risk of missing high-frequency motion details.

Safety of Template Augmentation vs. Diversity of LLMs: Considering the strict accuracy requirements for VLA instructions, we opted for a template-based rewriting strategy, sacrificing linguistic diversity in exchange for 100% semantic consistency.

Strict Aggregation (min) vs. Lenient Filtering (mean): When processing multi-trajectory motion scoring, using min aggregation maximizes the interception of anomalous trajectories (high precision), but it is also prone to incorrectly discarding high-value samples that contain minor, localized noise (low recall). This requires dynamic adjustment based on dataset scale during the actual construction of training sets.

6.6 Evaluation Validity

The validity of the current system evaluation methods can be examined across the following four dimensions:

Internal Validity: Through rigorous component ablation studies, we effectively validated the logical superiority of "dual-channel refinement" over "single-metric refinement."

External Validity: The current static benchmarks focus on synthetic images and five specific categories of robotic tasks. The system's generalization capabilities in handling extreme lighting conditions or open-vocabulary long-tail real-world scenarios require further empirical validation.

Construct Validity: Utilizing the ratio of jitter and jerk in motion trajectories as metrics for action data quality is theoretically sound in physics and control theory, effectively reflecting the smoothness and usability of the demonstration data.

Reproducibility: Although the code architecture is fully open-source and data formats are standardized, achieving completely consistent reproducibility across different time periods remains inherently challenging due to the reliance on external large model APIs.

6.7 Implications for VLA Data Pipelines

The engineering practice of Semantic-Motion offers several core insights for data engineering of future large-scale VLA models:

First, independent vision-language understanding validation is mandatory before feeding data into action generation models. Raw collected motion data contains significant amounts of unintentional wandering and trial-and-error; all trajectories cannot be blindly accepted.

Second, automatically generated task labels must never be used directly (i.e., "nakedly"). Pseudo-labels generated by large models must undergo a controlled feedback loop for rigorous review.

Finally, data refinement must move towards multimodal joint verification. Future filtering systems must evaluate not only whether the textual description matches the video (semantic dimension) but also whether the trajectory is physically smooth (physical dimension). Only samples where visual evidence and motion evidence corroborate each other can support the generalized learning of next-generation, high-performance robots.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project set out to close a specific engineering gap: the absence of a lightweight, local and auditable path from raw or rendered loco-manipulation motion to structured, quality-controlled Video–Motion–Text samples suitable as candidates for VLA research. The delivered system meets that aim as a modular and observable pipeline. The Rendering Module turns motion into synchronized fixed and egocentric videos with real 3D trajectories; the Perception Module converts synchronized views into temporally grounded macro intents and micro instructions through overlapping-window perception and weighted boundary voting; the Augmentation Module produces auditable instruction variants with source-step provenance; and the Refinement Module normalizes motion, scores physical quality, independently verifies text–video consistency and emits explainable decisions. The modules communicate through versioned, file-based contracts, and each stage retains an independent per-sample artifact so that a final decision can be traced backwards to the evidence that produced it.

The evidence is reported by class rather than compressed into a single headline number. Automatically verified behaviour covers the interface contracts, paired-view fusion, window aggregation, refinement structure, temporal and classification metrics, and the pinned upstream adapter. Controlled motion corruptions confirm that the physical-quality channel ranks clean above corrupted trajectories and localizes the failure type, and a weak LIBERO proxy indicates that multi-view fusion helps relative to a fixed view. The remaining evidence is explicitly conditional: semantic perception and judging depend on an external API, rendering depends on Blender, and formal boundary and keep/drop accuracy await human-adjudicated gold. In keeping with the discipline adopted throughout, no completion is inferred beyond what has been verified. The principal contribution is therefore architectural and engineering: a runnable and inspectable interface between motion rendering, video semantics, language generation and multimodal quality control.

7.2 Future Work

Several extensions follow directly from the limitations identified above.

- **Deterministic factual validation.** Close the FR5 gap by adding a post-generation fact gate for LLM-augmented text that verifies actor, object, action, order and goal-state preservation against the source annotation before a variant is admitted.
- **Human-adjudicated gold and re-enabled gates.** Complete the double-annotation and adjudication of the pending paired-view packets so that formal boundary, segment and keep/drop metrics can be reported, and re-enable the Module C quality gates once labelled thresholds can be validated.
- **Broader dataset coverage.** Scale the WGO-Bench and LIBERO subsets beyond the current integration size, and add adapters for further motion and trajectory formats, before making benchmark-wide claims.
- **Embodiment mapping.** Introduce an explicit alignment from human skeletal motion to robot

end-effector spaces to reduce the embodiment gap between rendered human demonstrations and robot action spaces.

- **Reproducibility and independent judging.** Support a locally hosted or version-pinned verifier from a different model family to reduce API non-determinism and to avoid coupling between the generation and judging models.
- **Motion scoring refinements.** Add severity-aware, peak-sensitive penalties and per-track weighting so that isolated but critical spikes are not diluted by averaging.
- **Durable orchestration.** Extend the observability layer with persistent task storage, authentication, restricted CORS and a dedicated job queue for multi-user, multi-worker deployment.

Bibliography

[1] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. doi: 10.15607/RSS.2023.XIX.026. URL https://www.roboticsproceedings.org/rss19/p026.html.

[2] Danny Driess, Fei Xia, Mehdi Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-E: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488, 2023. URL https://proceedings.mlr.press/v202/driess23a.html.

[3] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of the 8th Conference on Robot Learning*, volume 270, pages 2679–2713, 2024. URL https://mlanthology.org/corl/2024/kim2024corl-openvla/.

[4] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/8c3c666820ea055a77726d66fc7d447f-Abstract.html.

[5] Hao Luo, Ye Wang, Wanpeng Zhang, Sipeng Zheng, Ziheng Xi, Chaoyi Xu, Haiweng Xu, Haoqi Yuan, Chi Zhang, Yiqing Wang, Yicheng Feng, and Zongqing Lu. Being-H0.5: Scaling human-centric robot learning for cross-embodiment generalization. *arXiv preprint arXiv:2601.12993*, 2026. doi: 10.48550/arXiv.2601.12993. URL https://arxiv.org/abs/2601.12993.

[6] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1820–1864, 2023. URL https://proceedings.mlr.press/v229/mandlekar23a.html.

[7] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of household tasks for generalist robots. In *Proceedings of Robotics: Science and Systems*, 2024. doi: 10.15607/RSS.2024.XX.050. URL https://roboticsproceedings.org/rss20/p050.html.

[8] Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and RT-X models. In *2024 IEEE International Conference on Robotics and Automation*, pages 6892–6903, 2024. URL https://arxiv.org/abs/2310.08864.

[9] Video2Tasks Contributors. Video2Tasks: Split multi-task robot videos into single-task segments with auto-generated instructions. https://github.com/ly-geming/video2tasks. Open-source software repository, accessed 15 July 2026.

[10] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1723–1736, 2023. URL https://mlanthology.org/corl/2023/walke2023corl-bridgedata/.

[11] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag Sanketi, Grecia Salazar, Michael Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

Chapter A

VLM System Prompts

The Perception Module uses one structured system prompt (`src/prompts.py`) that requires the model to reason only over visible evidence and to return strict JSON, together with a per-window instruction assembled at run time. The system prompt is reproduced below.

```
You are a loco-manipulation task analyst. You will be given one or more
temporally ordered images from fixed and/or egocentric cameras together with an
optional task instruction. Analyze only visible evidence and output a JSON object
with the following fields:

1.  objects           - list of objects visible in the scene
2.  spatial_relations - list of spatial relationships, e.g. "red mug ON counter"
3.  task_type         - a concise locomotion or manipulation type such as walk,
                        sit, stand, reach, grasp, pick_and_place, open, close,
                        push, pull, rotate, turn_on, turn_off, carry, handover,
                        sweep, hang, measure, unplug, sort, move, or other
4.  domain            - locomotion, manipulation, mixed, or unknown
5.  action_sequence   - ordered list of primitive observed actions
6.  action_details    - one object per action with keys text, verb, object,
                        body_part, contact_state
                        (none|approach|contact|grasp|release|unknown),
                        posture, start_image_index, end_image_index
7.  target_object     - the main object involved, or empty for pure locomotion
8.  destination       - where the body/object should end up (null if n/a)
9.  instruction       - one evidence-grounded summary instruction
10. transitions       - image indices where one atomic task ends and another
                        begins
11. confidence        - confidence from 0 to 1

Do not infer hidden contact or trajectories. Use unknown when evidence is
insufficient. Respond with ONLY valid JSON. No explanation or markdown fences.
```

At run time the pipeline prepends the optional source instruction and appends a window-specific note; for fused input this note states that, at each timestamp, images are ordered fixed then ego and that transition indices refer to timestamps rather than to individual images.

Chapter B

Semantic Consistency Prompt

The Refinement Module verifies text–video consistency with an independent prompt (`src/module_c/semantic_consistency_v1.txt`). It evaluates seven dimensions and returns a normalized JSON verdict. The checklist and output contract are summarized below.

```
Evaluate every dimension separately before assigning the final label, using only
visible evidence:
1) Actor correctness       - who/what performs the action (gripper, hand, person)
2) Object correctness      - primary manipulated object and visible object part
3) Action correctness      - concrete interaction; attempted vs. completed
4) Temporal correctness    - order of approach, contact, grasp, move, place
5) Goal/final-state        - destination, spatial relation, achieved outcome
6) Hallucination check     - any claim not supported by visible evidence
7) Evidence sufficiency    - visibility, occlusion, blur, sampling gaps

Decision priority: a clear contradiction in any core claim -> inconsistent;
consistent only when all core claims are visibly supported; uncertain when
evidence is incomplete or ambiguous.

Output (JSON only):
  label          : one of ["consistent", "uncertain", "inconsistent"]
  confidence     : float in [0, 1]   (confidence in the label, not a score)
  error_types    : subset of [actor_mismatch, object_mismatch, action_mismatch,
                   temporal_mismatch, goal_state_mismatch, possible_hallucination,
                   insufficient_evidence, low_detail, manipulation_irrelevant]
  suggested_text : concise "<verb> <object> [to/on/in/from <goal>]" summary

The filtering pipeline keeps a sample only when label == "consistent".
```

Chapter C

Data Schemas

Upstream contracts are Pydantic models in `src/semantic_motion/models.py`; the refinement input/output are dataclasses in `src/module_c/schema.py`. The core contracts are reproduced in condensed form below.

```
ViewBundle (semantic-motion-view-bundle/v1)
  sample_id, timebase: SharedTimebase, views: {view_id -> ViewStream},
  trajectory: MotionReference | None, provenance: Provenance

SharedTimebase
  fps, frame_count, duration_sec, frame_map: FrameMap

ViewStream
  view_id, video_path, camera_type, fps, frame_count, duration_sec

MotionReference
  path, source, spatial_unit, coordinate_frame, track_names

MacroIntent
  task_type, target_object, destination, confidence,
  domain in {locomotion, manipulation, mixed, unknown}

MicroInstruction
  step_id, text, verb, object, confidence, body_part, contact_state,
  posture, observed_trajectory: ObservedTrajectoryRef | None,
  evidence_frame_ids

TaskSegment
  segment_id, start/end_time_sec, start/end_frame, frame_ids, task_instruction,
  objects, spatial_relations, macro_intent, micro_instructions,
  augmented_instructions, confidence, evidence_by_view, metadata

VideoTaskRecord (semantic-motion-video-task/v2)
  video_path, view_bundle, source_instruction, multi_view_frames,
  task_segments, metadata

Sample (Module C)         : sample_id, views/videos, text, motion tracks,
                            interval, provenance
RefinementResult          : motion_quality_score, semantic_label,
                            semantic_confidence, decision, reason_codes,
                            auxiliary diagnostics
```

Chapter D

Configuration

Deterministic thresholds and verifier settings are configuration-driven. The default Module C configuration (`configs/module_c_default.yaml`) is reproduced below.

```yaml
thresholds:
  motion_min: 0.35
  semantic_min_confidence: 0.70

motion_quality:
  max_velocity: 2.5
  max_acceleration: 6.0
  max_jerk: 20.0
  max_jitter_ratio: 0.30
  max_interval_cv: 0.25
  max_time_gap_ratio: 2.5
  aggregation: min

sync:
  require_paired_views: true
  fps_tolerance: 0.001
  duration_tolerance_sec: 0.05
  require_motion_coverage: true

policy:
  allow_mock_keep: false

semantic:
  verifier: qwen3-vl-flash
  qwen_vl_base_url: "https://dashscope.aliyuncs.com/compatible-mode/v1"
  qwen_vl_api_key_env: DASHSCOPE_API_KEY
  qwen_vl_model: qwen3-vl-flash
  request_timeout_s: 300.0
  prompt_file: src/module_c/semantic_consistency_v1.txt
  local_video_max_upload_mb: 18.0
  local_video_frame_count: 8
  local_video_frame_jpeg_quality: 85
```

Chapter E

Additional Results

The principal quantitative tables are given in Chapter 5: Table 5.2 (LIBERO title-as-single-segment weak proxy), Table 5.3 (controlled motion corruptions) and Table 5.4 (engineering acceptance). This appendix records the supporting pinned-baseline comparison and the corresponding artifacts.

Table E.1. Pinned prompt ablation on the WGO-Bench robot subset with human timestamped labels (Qwen3-VL-Flash, one episode × three repeats, identical windowing and aggregation). Integration-scale only; not a superiority claim.

Prompt arm	Instruction token F1
Pinned upstream Video2Tasks prompt	0.200
Semantic-Motion blind prompt	0.116

Result artifacts are retained under `results/`, including:

- `results/libero_goal_title_as_single_segment.json`
- `results/motion_corruption_benchmark.json`
- `results/wgo_video2tasks_prompt_ablation.json`
- `results/eval_20260401_175616.json` (static diagnostic run)

Each records the code revision, model, endpoint and pricing metadata used, and marks whether its numbers are formal or weak-proxy.

Chapter F

User Guide

The pipeline is driven by command-line entry points; the Web layer wraps the same commands. Representative invocations for each module follow.

```
# Module D - render motion to synchronized multi-view video + trajectories
blender your_scene.blend --background --python module_d/render_pipeline.py

# Module A - paired-view video-to-task perception (fixed | ego | fused)
python run_video_task.py \
  --manifest data/libero_goal/processed/manifest.json \
  --sample-id put_the_bowl_on_the_plate_demo_0 \
  --view-mode fused --model qwen3-vl-flash --rewriter llm \
  --max-frames 16 --variants 3

# Modules A+B+C - one-shot generation, sample preparation and refinement
python run_generate_filter.py \
  --manifest data/libero_goal/processed/manifest.json \
  --sample-id put_the_bowl_on_the_plate_demo_0 --view-mode fused \
  --vlm-model qwen3-vl-flash --semantic-verifier qwen3-vl-flash \
  --refine-config configs/module_c_default.yaml --sample-level segment

# Pinned Video2Tasks prompt ablation (shared windowing and aggregation)
python compare_video2tasks.py --mode prompt-ablation \
  --model qwen3-vl-flash --output results/video2tasks_prompt_ablation.json

# Evaluation utilities
python tools/evaluate_libero_goal.py --views all --repeats 3 --max-frames 16
python tools/evaluate_motion_corruptions.py \
  --motion data/libero_goal/processed/TASK/demo_0_traj.json
```

A DashScope-compatible API key must be provided through the environment (`DASHSCOPE_API_KEY`, `DASHSCOPE_BASE_URL`, `VLM_MODEL`); credentials are never stored in source. Module D additionally requires a Blender installation and a scene with the expected camera and ground-plane object names.